Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase II · Aprendizaje automático y redes neuronales

Módulo 10

Diferenciación automática

De la regla de la cadena a `loss.backward()`

Apuntes del curso

Edición de octubre de 2026

Presentación del módulo

Un paso de entrenamiento de un modelo de lenguaje exige la derivada de la pérdida respecto a cada uno de sus parámetros. El modelo Grok-1, cuyos pesos publicó xAI en 2024, tiene 314 000 millones; calcular cada derivada por diferencias finitas requeriría otras tantas evaluaciones de la red. En el módulo 9 se obtuvieron a mano las ecuaciones de la retropropagación de un perceptrón de dos capas, pero ese procedimiento no escala: cada cambio de arquitectura obligaría a deducirlas de nuevo. La *diferenciación automática* convierte la regla de la cadena en un algoritmo que se aplica a cualquier programa compuesto de operaciones elementales y que devuelve el gradiente exacto, salvo errores de redondeo, con un coste comparable al de evaluar la función una sola vez. Este módulo estudia sus dos modos, demuestra por qué el modo inverso es el adecuado para entrenar redes, analiza su coste en operaciones y en memoria, y termina con la implementación de un pequeño sistema de diferenciación automática y con su contrapartida industrial, PyTorch.

Objetivos

Al finalizar el módulo, el estudiante debe ser capaz de:

1. representar un cálculo como un grafo computacional y como una traza de evaluación, y distinguir la diferenciación automática de la numérica y de la simbólica, acotando el error de las diferencias finitas;
2. definir los números duales, demostrar que propagan exactamente la derivada y aplicar el modo directo para obtener productos jacobiano-vector;
3. enunciar y demostrar la corrección del modo inverso, construir un orden topológico y retropropagar a mano por un grafo arbitrario;
4. demostrar el principio del gradiente barato, comparar el coste de los dos modos mediante la asociatividad del producto de jacobianos y deducir los productos vector-jacobiano de la capa lineal y de la softmax, así como la regla $C \approx 6ND$;
5. analizar la memoria del modo inverso y el recálculo por puntos de control, obtener productos hessiana-vector e interpretar los adjuntos como multiplicadores de Lagrange;
6. describir la estructura de un sistema de diferenciación automática escalar y de PyTorch, y usar correctamente tensores, `backward`, `nn.Module` y los optimizadores.

Requisitos previos

Del módulo 4 se usan la regla de la cadena multivariable, el jacobiano, el hessiano, el desarrollo de Taylor y las diferencias centradas; del módulo 7, el descenso de gradiente, el decaimiento de pesos y los multiplicadores de Lagrange; del módulo 8, la entropía cruzada con softmax, y del módulo 9, el perceptrón multicapa, su contador de parámetros y las ecuaciones de la

retropropagación. Del módulo 1 se usa la softmax y su gradiente. Se suponen nociones básicas de programación en Python.

Organización

Tema	Contenido	Fuentes principales
1	Grafos computacionales y métodos de diferenciación	[2], [10], [9], [17]
2	El modo directo y los números duales	[2], [10], [17], [18]
3	El modo inverso y la retropropagación	[2], [10], [9], [5], [23]
4	Coste del gradiente y productos vector-jacobiano	[10], [2], [12], [13]
5	Memoria, segundo orden y método adjunto	[10], [3], [20], [7]
6	Implementación: un autograd escalar y PyTorch	[14], [19], [21], [4]

Cada tema termina con una lista de lecturas y con ejercicios graduados en tres niveles: $\star$ comprensión, $\star\star$ cálculo y $\star\star\star$ demostración. Los ejercicios marcados con *Fuente* están adaptados de cursos y libros de texto universitarios; el enunciado se ha redactado de nuevo y, en algunos casos, se han cambiado los datos para que puedan resolverse a mano. Las soluciones completas, paso a paso, están en el apéndice A.

Sobre las fuentes

El texto es de redacción propia. La estructura general sigue el artículo de revisión de Baydin, Pearlmutter, Radul y Siskind [2], en particular sus secciones 2 y 3, y la notación de la traza de evaluación es la del libro de Griewank y Walther [10], texto de referencia de la disciplina, del que se toman también las cotas de coste (cap. 4) y el recálculo por puntos de control (cap. 12). El punto de vista de las redes neuronales procede de la §6.5 de Goodfellow, Bengio y Courville [9], de las notas «Backpropagation, Intuitions» de CS231n [5], de las §2.5 y §5.3 de Zhang *et al.* [23] y de la lección 6 del curso CSC421 de Toronto [12]. Los números duales se presentan como en las notas del curso 18.337 del MIT [17] y en las clases de 18.S096 [18]. Los problemas de coste y de memoria de los temas 4 y 5 están adaptados de las tareas 1 y 2 del curso CS336 de Stanford [6, 7], y la implementación del tema 6 sigue a micrograd [14], a la tarea complementaria de autograd del curso 11-785 de Carnegie Mellon [4], al artículo de PyTorch [19] y a su documentación [21]. Sobre los modelos comerciales (Claude, Gemini, Grok) solo se usa información pública.

Notación

Símbolo	Significado
$f\colon \mathbb{R}^n \to \mathbb{R}^m$	función que se deriva, con n entradas y m salidas
$v_{1-n}, \dots, v_0$	variables de entrada de la traza de evaluación
$v_1, \dots, v_\ell$	variables intermedias, $v_i = \varphi_i(v_j)_{j \prec i}$
$j \prec i$	v_j es un argumento de la operación elemental φ_i
$\dot{v}_i$	tangente: derivada de v_i en una dirección fijada de las entradas
$\bar{v}_i = \partial y / \partial v_i$	adjunto de v_i respecto a la salida escalar y
$J_f(x) \in \mathbb{R}^{m \times n}$	jacobiano de f en x
$\mathbb{D}$, $a + b\varepsilon$	números duales, con $\varepsilon^2 = 0$
$\mathrm{ops}(f)$	número de operaciones aritméticas para evaluar f
N, D	número de parámetros de un modelo y de tokens de entrenamiento
$\mathbf{u}$	unidad de redondeo de la aritmética en coma flotante

Como en el módulo 9, los vectores que fluyen por una red se escriben como *filas* y las capas actúan por la derecha, $X \mapsto XW$; los gradientes respecto a una matriz tienen la forma de la matriz. En los temas 2 y 3, en cambio, el jacobiano actúa sobre vectores columna, $\dot{y} = J\dot{x}$, como en [2] y [10]. El código se escribe en Python y los nombres de PyTorch aparecen en `letra mecanográfica`.

Índice

Presentación del módulo **ii**

1 Grafos computacionales y métodos de diferenciación **1**
1.1 Trazas de evaluación . . . 1
1.2 Tres maneras de derivar . . . 2
1.2.1 Diferencias finitas . . . 2
1.2.2 Derivación simbólica . . . 3
1.2.3 Diferenciación automática . . . 3
Ejercicios del tema 1 . . . 4

2 El modo directo y los números duales **6**
2.1 Propagación de tangentes . . . 6
2.2 Números duales . . . 7
2.3 Coste del modo directo . . . 8
Ejercicios del tema 2 . . . 9

3 El modo inverso y la retropropagación **10**
3.1 Adjuntos . . . 10
3.2 El algoritmo . . . 11
3.3 El orden topológico . . . 13
Ejercicios del tema 3 . . . 14

4 Coste del gradiente y productos vector-jacobiano **15**
4.1 El principio del gradiente barato . . . 15
4.2 Asociatividad del producto de jacobianos . . . 16
4.3 Productos vector-jacobiano . . . 16
4.4 El cómputo de un paso de entrenamiento . . . 17
Ejercicios del tema 4 . . . 18

5 Memoria, segundo orden y método adjunto **20**
5.1 La memoria del modo inverso . . . 20
5.2 Recálculo por puntos de control . . . 20
5.3 Productos hessiana-vector . . . 21
5.4 Los adjuntos como multiplicadores de Lagrange . . . 22
Ejercicios del tema 5 . . . 23

6 Implementación: un autograd escalar y PyTorch **24**
6.1 Un autograd escalar . . . 24
6.2 PyTorch . . . 26
6.3 Acumulación de gradientes . . . 27
Ejercicios del tema 6 . . . 28

A Soluciones de los ejercicios **29**
A.1 Tema 1 . . . 29
A.2 Tema 2 . . . 31
A.3 Tema 3 . . . 34
A.4 Tema 4 . . . 36
A.5 Tema 5 . . . 39
A.6 Tema 6 . . . 41

Bibliografía **44**

Tema 1

Grafos computacionales y métodos de diferenciación

Este tema fija el objeto que se deriva: un programa que calcula una función a partir de operaciones elementales. Se formaliza como traza de evaluación y como grafo dirigido acíclico, y se comparan las tres maneras de obtener derivadas con un ordenador: diferencias finitas, derivación simbólica y diferenciación automática. Se siguen las secciones 2 y 3 de [2], el capítulo 2 de [10], la §6.5.1 de [9] y las notas de la clase 8 de MIT 18.337 [17].

1.1 Trazas de evaluación

Todo programa que calcula un número real a partir de otros sin bifurcaciones que dependan de los datos ejecuta una sucesión finita de operaciones aritméticas y de funciones de biblioteca (exp, ln, sen, tanh…). Cuando hay bifurcaciones o bucles, cada ejecución concreta recorre una sucesión de este tipo, que depende de la entrada; las derivadas se toman respecto a esa sucesión.

Definición 1.1 (Operaciones elementales). Un conjunto de *operaciones elementales* es una colección finita Φ de funciones $\varphi\colon U_\varphi \subseteq \mathbb{R}^{k_\varphi} \to \mathbb{R}$, con U_φ abierto y $k_\varphi \le k$ argumentos, de clase C^1 y cuyas derivadas parciales $\partial\varphi/\partial u_j$ se pueden evaluar con un número acotado de operaciones. En este módulo Φ contiene la suma, la resta, el producto, la división, las potencias de exponente fijo, exp, ln, sen, cos y tanh.

Definición 1.2 (Traza de evaluación). Sea $f\colon \mathbb{R}^n \to \mathbb{R}^m$. Una *traza de evaluación* de f es una sucesión de variables

$$v_{1-n}, \dots, v_0,\ v_1, \dots, v_\ell$$

tal que $v_{i-n} = x_i$ para $i = 1, \dots, n$, cada variable intermedia se obtiene como $v_i = \varphi_i\big((v_j)_{j \prec i}\big)$ con $\varphi_i \in \Phi$ y $j \prec i \Rightarrow j < i$, y las salidas son $y_{m-i} = v_{\ell-i}$ para $i = 0, \dots, m-1$. La relación $j \prec i$ se lee «v_j es un argumento de φ_i».

La condición $j < i$ expresa que cada variable se calcula a partir de variables ya calculadas. Es la notación de Griewank y Walther [10, cap. 2], que Baydin *et al.* adoptan en [2, §3]: las entradas llevan índices no positivos, las intermedias, positivos, y las salidas son las últimas variables.

Definición 1.3 (Grafo computacional). El *grafo computacional* de una traza es el grafo dirigido $G = (V, E)$ con un nodo por variable y una arista $v_j \to v_i$ por cada par $j \prec i$. Los nodos sin aristas entrantes son las entradas.

Proposición 1.4. *El grafo computacional de una traza es acíclico, y el orden de la traza es un* orden topológico*: toda arista va de un índice menor a uno mayor.*

Demostración. Por la definición 1.2, toda arista $v_j \to v_i$ cumple $j < i$. Un ciclo $v_{i_1} \to v_{i_2} \to \cdots \to v_{i_1}$ daría $i_1 < i_2 < \cdots < i_1$, lo cual es imposible. ∎

Ejemplo 1.5 (El ejemplo de Baydin *et al.*). Sea $f(x_1, x_2) = \ln x_1 + x_1 x_2 - \operatorname{sen} x_2$, evaluada en $(x_1, x_2) = (2, 5)$ [2, §3, fig. 4 y tabla 2]. Su traza es

Variable	Definición	Valor
v_{-1}	x_1	2
v_0	x_2	5
v_1	$\ln v_{-1}$	0,6931
v_2	$v_{-1} \cdot v_0$	10
v_3	$\operatorname{sen} v_0$	−0,9589
v_4	$v_1 + v_2$	10,6931
v_5	$v_4 - v_3$	11,6521

y $y = v_5$. La figura 1.1 muestra el grafo: v_{-1} alimenta a dos operaciones (v_1 y v_2), igual que v_0 (v_2 y v_3).

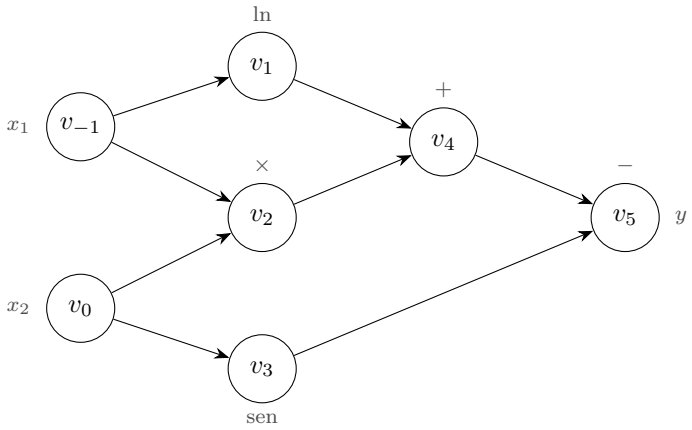


Figura 1.1: Grafo computacional de la traza del ejemplo 1.5. Cada nodo interior lleva la operación elemental que lo calcula.

Observación 1.6. En un modelo de lenguaje, la traza de una pasada hacia delante tiene miles de millones de operaciones escalares, pero se organiza en unas pocas operaciones sobre tensores: productos de matrices, softmax, normalizaciones. El grafo que construyen los sistemas reales tiene un nodo por operación tensorial (tema 6); la teoría de los temas 2 a 5 se formula con escalares y se extiende sin cambios a bloques.

1.2 Tres maneras de derivar

1.2.1 Diferencias finitas

La aproximación más directa sustituye la derivada por un cociente incremental. Para $f\colon \mathbb{R}^n \to \mathbb{R}$ y la base canónica $e_1, \ldots, e_n$,

$$\frac{\partial f}{\partial x_i}(x) \approx \frac{f(x + h e_i) - f(x)}{h} \qquad \text{o} \qquad \frac{\partial f}{\partial x_i}(x) \approx \frac{f(x + h e_i) - f(x - h e_i)}{2h}.$$

En el módulo 4 (tema 6) se demostró que el error de truncamiento es $O(h)$ en la primera fórmula y $O(h^2)$ en la segunda, y que el redondeo añade un error de orden $\mathbf{u}|f|/h$, donde $\mathbf{u} \approx 1{,}1 \cdot 10^{-16}$ es la unidad de redondeo de doble precisión. El compromiso entre ambos da un paso óptimo $h^* \sim \mathbf{u}^{1/3}$ para la diferencia centrada, con un error mínimo de orden $\mathbf{u}^{2/3} \approx 10^{-11}$.

Proposición 1.7 (Coste de las diferencias finitas). *Aproximar el gradiente de $f\colon \mathbb{R}^n \to \mathbb{R}$ por diferencias centradas requiere $2n$ evaluaciones de f; por diferencias hacia delante, $n+1$.*

Demostración. Cada componente centrada usa $f(x + he_i)$ y $f(x - he_i)$, distintas para cada i: $2n$ evaluaciones. Hacia delante, $f(x)$ es común a todas las componentes y cada una añade $f(x + he_i)$: $n + 1$. ■

Con $n = 10^{11}$ parámetros, ni la precisión ni el coste son aceptables. Las diferencias finitas se usan para *comprobar* un gradiente en unas pocas direcciones, nunca para entrenar [2, §2.1].

Observación 1.8 (Paso complejo). Si f es analítica y real sobre la recta real, el desarrollo de Taylor $f(x + ih) = f(x) + ihf'(x) - \frac{1}{2}h^2 f''(x) - \frac{i}{6}h^3 f'''(x) + \dots$ da

$$f'(x) = \frac{\operatorname{Im} f(x + ih)}{h} + O(h^2).$$

No hay resta de cantidades próximas, de modo que el error de redondeo no crece al disminuir h y puede tomarse $h = 10^{-20}$. Es un precursor del modo directo del tema 2: la unidad imaginaria hace el papel de un infinitesimal, aunque con $i^2 = -1$ en lugar de $\varepsilon^2 = 0$ [17].

1.2.2 Derivación simbólica

La derivación simbólica aplica las reglas del cálculo a la *expresión* de f y devuelve otra expresión. Es exacta, pero, si se aplica sin compartir subexpresiones, el tamaño del resultado puede crecer exponencialmente [2, §2.2].

Ejemplo 1.9 (Crecimiento de las expresiones). Sea la recurrencia logística $l_1 = x$, $l_{k+1} = 4l_k(1 - l_k)$ [2, tabla 1]. Se tiene $l_2 = 4x(1 - x)$ y $l_3 = 16x(1 - x)(1 - 2x)^2$. Por la regla del producto, la derivada de l_{k+1} contiene la de l_k y una copia de l_k: $l'_{k+1} = 4l'_k(1 - l_k) - 4l_k l'_k$. Si cada l_k se sustituye por su expresión en x, la expresión de l'_k duplica al menos su tamaño en cada paso. En cambio, el par (l_k, l'_k) se puede calcular numéricamente con un número constante de operaciones por paso, como se verá en el tema 2.

1.2.3 Diferenciación automática

Definición 1.10 (Diferenciación automática). La *diferenciación automática* (*automatic differentiation*, AD) es la familia de técnicas que calculan valores numéricos de derivadas de una función definida por un programa, aplicando la regla de la cadena a los *valores* de la traza de evaluación y usando, en cada operación elemental, su derivada local conocida.

Como subraya [2, §3], la AD no manipula expresiones sino números: cada operación elemental aporta una derivada local que se combina con las demás. A diferencia de las diferencias finitas, el resultado es exacto salvo el redondeo de cada operación; a diferencia de la derivación simbólica, el coste es proporcional al de la traza. La regla de la cadena admite dos órdenes de aplicación, que dan los dos modos de la AD: el *directo* (*forward*), del tema 2, y el *inverso* (*reverse*), del tema 3.

Método	Exactitud	Coste de ∇f, $f\colon \mathbb{R}^n \to \mathbb{R}$	Uso
Diferencias finitas	error $O(h^2)$ más redondeo	$2n$ evaluaciones	comprobar
Simbólica	exacta	expresiones que pueden crecer	cálculo a mano
AD, modo directo	exacta salvo redondeo	n pasadas	pocas entradas
AD, modo inverso	exacta salvo redondeo	una pasada (teorema 4.1)	entrenar redes

Observación 1.11 (Conexión con los LLM). La retropropagación de los módulos 8 y 9 es el modo inverso de la AD aplicado a la traza de una red neuronal [9, §6.5]. Una llamada a `loss.backward()` en PyTorch recorre la traza registrada durante la pasada hacia delante y aplica en cada nodo la regla local que se estudia en el tema 3.

Lecturas recomendadas para este tema

- Baydin *et al.* [2], §2 «What AD Is Not» (§2.1 y §2.2) y comienzo de la §3, con la figura 4 y la tabla 2.
- Griewank y Walther [10], cap. 2 «A Framework for Evaluating Functions».
- Goodfellow, Bengio y Courville [9], §6.5.1 «Computational Graphs».
- MIT 18.337 [17], notas de la clase 8, apartados «Finite Differencing and Numerical Stability» y «Complex Step Differentiation».
- Grosse [12], lección 6, apartados iniciales sobre las alternativas a la AD.

Ejercicios del tema 1

Ejercicio 1.1 ⋆ Para el ejemplo 1.5, indique cuántos nodos y aristas tiene el grafo, qué nodos tienen más de un sucesor y escriba otro orden topológico distinto del de la traza.

Ejercicio 1.2 ⋆ Ordene de menor a mayor precisión y justifique: diferencias centradas con $h = 10^{-5}$, diferenciación automática y diferencias centradas con $h = 10^{-15}$, todo en doble precisión. Ilústrelo con $f(x) = e^x$ en $x = 1$.

Ejercicio 1.3 ⋆⋆ Sea $f(x) = \ln(x^2) \cdot \operatorname{sen} x + x^{-1}$. (a) Descomponga f en operaciones elementales y dibuje su grafo de dependencias, de x a $f(x)$. (b) Escriba la traza de evaluación en $x = 2$ (radianes) con cuatro cifras significativas.

Fuente: adaptado de Zhang *et al.* [23], §2.5 «Automatic Differentiation», ejercicio 5.

Ejercicio 1.4 ⋆⋆ Para $f(x) = e^x$, demuestre que $\operatorname{Im} f(x + ih)/h = e^x \operatorname{sen}(h)/h$ y deduzca que el error relativo de la aproximación por paso complejo es $1 - \operatorname{sen}(h)/h \approx h^2/6$. Calcúlelo para $h = 10^{-1}$ y $h = 10^{-8}$ y compárelo con el de la diferencia centrada con el mismo h en doble precisión, cuyo error relativo es $\sinh(h)/h - 1$ más un término de redondeo de orden $\mathbf{u}/h$.

Fuente: adaptado de MIT 18.337 [17], notas de la clase 8, apartado «Complex Step Differentiation».

Ejercicio 1.5 ⋆⋆ Para la recurrencia del ejemplo 1.9 con $x = 0{,}2$, calcule l_2, l_3 y las derivadas l_2', l_3' propagando el par (l_k, l_k') con la regla $l_{k+1}' = 4l_k'(1 - 2l_k)$. Compruebe el resultado derivando la expresión $l_3 = 16x(1 - x)(1 - 2x)^2$.

Fuente: adaptado de Baydin *et al.* [2], §2.2 y tabla 1 (se añade el cálculo numérico).

Ejercicio 1.6 ⋆⋆⋆ Para la recurrencia del ejemplo 1.9, demuestre que l_k es un polinomio en x de grado exactamente 2^{k-1}, de modo que su expresión desarrollada tiene hasta $2^{k-1}+1$ coeficientes, y que, en cambio, el par (l_k, l'_k) se obtiene con a lo sumo $6(k-1)$ operaciones aritméticas a partir de $(x, 1)$.

Tema 2

El modo directo y los números duales

El modo directo propaga, junto con cada valor de la traza, su derivada respecto a una dirección fijada de las entradas. Este tema lo formula como propagación de tangentes, demuestra que calcula un producto jacobiano-vector y lo reinterpreta con los números duales, que reducen la regla de la cadena a aritmética. Se siguen la §3.1 de [2], incluida la §3.1.1 sobre números duales, el capítulo 3 de [10], las notas de la clase 8 de MIT 18.337 [17] y la clase 5 de MIT 18.S096 [18].

2.1 Propagación de tangentes

Se fija una traza de $f\colon \mathbb{R}^n \to \mathbb{R}^m$ (definición 1.2), un punto x y una dirección $\dot{x} \in \mathbb{R}^n$.

Definición 2.1 (Tangentes). Las *tangentes* de la traza son los números definidos por recurrencia como

$$\dot{v}_{i-n} = \dot{x}_i \quad (i = 1, \dots, n), \qquad \dot{v}_i = \sum_{j \prec i} \frac{\partial \varphi_i}{\partial v_j}\, \dot{v}_j \quad (i = 1, \dots, \ell),$$

donde las derivadas parciales se evalúan en los valores de la traza. El vector $\dot{x}$ se llama *semilla*.

Teorema 2.2 (El modo directo calcula $J\dot{x}$). *Sea $\gamma(t) = x + t\dot{x}$. Para cada i, $\dot{v}_i = \frac{d}{dt} v_i(\gamma(t))\big|_{t=0} = \nabla v_i(x)^\top \dot{x}$, donde $v_i(\cdot)$ denota la variable i como función de las entradas. En particular, el vector de tangentes de las salidas es $\dot{y} = J_f(x)\,\dot{x}$.*

Demostración. Por inducción sobre i. Para las entradas, $v_{i-n}(\gamma(t)) = x_i + t\dot{x}_i$, cuya derivada en $t = 0$ es $\dot{x}_i$. Sea $i \geq 1$ y supóngase el resultado cierto para todo índice menor; en particular, para cada $j \prec i$, porque $j < i$. Como $v_i(\gamma(t)) = \varphi_i\big((v_j(\gamma(t)))_{j \prec i}\big)$, la regla de la cadena (módulo 4) da

$$\frac{d}{dt} v_i(\gamma(t))\Big|_{t=0} = \sum_{j \prec i} \frac{\partial \varphi_i}{\partial v_j}\, \frac{d}{dt} v_j(\gamma(t))\Big|_{t=0} = \sum_{j \prec i} \frac{\partial \varphi_i}{\partial v_j}\, \dot{v}_j = \dot{v}_i.$$

Para las salidas, $\frac{d}{dt} f(x + t\dot{x})|_{t=0} = J_f(x)\dot{x}$ es la derivada direccional. ∎

Ejemplo 2.3 (Traza tangente). En el ejemplo 1.5, con semilla $\dot{x} = (1, 0)^\top$ se obtiene $\partial y / \partial x_1$ [2, tabla 2]:

Tangente	Regla	Valor
$\dot v_{-1}$, $\dot v_0$	semilla	1, 0
$\dot v_1$	$\dot v_{-1}/v_{-1}$	0,5
$\dot v_2$	$\dot v_{-1}v_0 + v_{-1}\dot v_0$	5
$\dot v_3$	$\cos(v_0)\,\dot v_0$	0
$\dot v_4$	$\dot v_1 + \dot v_2$	5,5
$\dot v_5$	$\dot v_4 - \dot v_3$	5,5

En efecto, $\partial f/\partial x_1 = 1/x_1 + x_2 = 0{,}5 + 5 = 5{,}5$. Con la semilla $(0,1)^\top$ se obtendría $\partial f/\partial x_2 = x_1 - \cos x_2 = 2 - \cos 5 \approx 1{,}7163$.

Corolario 2.4 (Coste del jacobiano en modo directo). *Una pasada en modo directo da una columna de $J_f(x)$, la correspondiente a la semilla e_k. El jacobiano completo de $f\colon \mathbb{R}^n \to \mathbb{R}^m$ requiere n pasadas, independientemente de m.*

Demostración. Por el teorema 2.2, con $\dot x = e_k$ se obtiene $J_f(x)e_k$, que es la columna k. Las n semillas $e_1, \dots, e_n$ dan las n columnas. ∎

El corolario fija el campo de aplicación del modo directo: es óptimo cuando hay pocas entradas y muchas salidas y es inviable para el gradiente de una pérdida escalar respecto a 10^{11} parámetros, que exigiría 10^{11} pasadas.

2.2 Números duales

Definición 2.5 (Números duales). El conjunto de los *números duales* es $\mathbb{D} = \{a + b\varepsilon : a, b \in \mathbb{R}\}$, con la suma y el producto

$$(a + b\varepsilon) + (c + d\varepsilon) = (a + c) + (b + d)\varepsilon, \qquad (a + b\varepsilon)(c + d\varepsilon) = ac + (ad + bc)\varepsilon.$$

El número a es la *parte real* y b, la *parte dual*.

El producto es el que resulta de multiplicar formalmente e imponer $\varepsilon^2 = 0$ con $\varepsilon \neq 0$. Con estas operaciones $\mathbb{D}$ es un anillo conmutativo con unidad, isomorfo a $\mathbb{R}[\varepsilon]/(\varepsilon^2)$, pero no es un cuerpo: $\varepsilon \cdot \varepsilon = 0$ con $\varepsilon \neq 0$, de modo que ε no tiene inverso [2, §3.1.1].

Proposición 2.6 (Inverso de un dual). *Si $a \neq 0$, el dual $a + b\varepsilon$ es invertible y*

$$\frac{1}{a + b\varepsilon} = \frac{1}{a} - \frac{b}{a^2}\,\varepsilon.$$

Demostración. $(a + b\varepsilon)\big(\frac{1}{a} - \frac{b}{a^2}\varepsilon\big) = 1 + \big(-\frac{b}{a} + \frac{b}{a}\big)\varepsilon = 1$. Si $a = 0$, $(b\varepsilon)(c + d\varepsilon) = bc\,\varepsilon \neq 1$ para todo c, d, y no hay inverso. ∎

La parte dual del inverso es b veces la derivada de $u \mapsto 1/u$ en a, y la parte dual del producto es la regla de Leibniz. Esto no es casual, y motiva la definición siguiente.

Definición 2.7 (Extensión dual de una operación elemental). Sea $\varphi\colon U \subseteq \mathbb{R}^k \to \mathbb{R}$ de clase C^1. Su *extensión dual* es

$$\varphi(a_1 + b_1\varepsilon, \dots, a_k + b_k\varepsilon) = \varphi(a) + \Big(\sum_{j=1}^{k} \frac{\partial \varphi}{\partial u_j}(a)\, b_j\Big)\varepsilon, \qquad a \in U.$$

Por ejemplo, $\exp(a + b\varepsilon) = e^a + e^a b\,\varepsilon$ y $\operatorname{sen}(a + b\varepsilon) = \operatorname{sen} a + (\cos a)\, b\,\varepsilon$.

Proposición 2.8 (Coherencia con la aritmética). *Para la suma, el producto y la inversión, la extensión dual de la definición 2.7 coincide con las operaciones de la definición 2.5 y de la proposición 2.6.*

Demostración. Para $\varphi(u_1, u_2) = u_1 + u_2$, las parciales valen 1 y la extensión da $(a_1 + a_2) + (b_1 + b_2)\varepsilon$. Para $\varphi(u_1, u_2) = u_1 u_2$, las parciales son a_2 y a_1, y la extensión da $a_1 a_2 + (a_2 b_1 + a_1 b_2)\varepsilon$, que es el producto de la definición 2.5. Para $\varphi(u) = 1/u$, $\varphi'(a) = -1/a^2$ y la extensión da la fórmula de la proposición 2.6. ∎

Teorema 2.9 (Los duales propagan la derivada). *Sea $f\colon \mathbb{R}^n \to \mathbb{R}^m$ dada por una traza y evalúese la traza en aritmética dual, con entradas $x_k + \dot{x}_k\varepsilon$ y la extensión dual de cada operación elemental. Entonces cada variable vale $v_i + \dot{v}_i\varepsilon$, con $\dot{v}_i$ la tangente de la definición 2.1, y las salidas valen*

$$f(x + \dot{x}\,\varepsilon) = f(x) + J_f(x)\,\dot{x}\,\varepsilon.$$

Demostración. Por inducción sobre i. Para las entradas es la elección de la semilla. Si las variables v_j con $j \prec i$ valen $v_j + \dot{v}_j\varepsilon$, la definición 2.7 da $\varphi_i\big((v_j + \dot{v}_j\varepsilon)_{j\prec i}\big) = \varphi_i\big((v_j)_{j\prec i}\big) + \big(\sum_{j\prec i} \frac{\partial \varphi_i}{\partial v_j}\dot{v}_j\big)\varepsilon = v_i + \dot{v}_i\varepsilon$. La última afirmación es el teorema 2.2. ∎

El teorema justifica la lectura del desarrollo de Taylor del módulo 4 con un infinitesimal: $f(a + b\varepsilon) = f(a) + f'(a)b\varepsilon$ es el desarrollo de primer orden, y aquí el truncamiento es exacto porque los términos de orden superior contienen $\varepsilon^2 = 0$. Para evaluar $f'(a)$ basta con calcular $f(a + \varepsilon)$ y leer la parte dual.

Ejemplo 2.10. Sea $f(x) = x^3 + 2x$ en $x = 2$. Con $x = 2 + \varepsilon$:

1. $x^2 = (2 + \varepsilon)(2 + \varepsilon) = 4 + 4\varepsilon$;
2. $x^3 = (4 + 4\varepsilon)(2 + \varepsilon) = 8 + (4 + 8)\varepsilon = 8 + 12\varepsilon$;
3. $2x = 4 + 2\varepsilon$, y $f = 12 + 14\varepsilon$.

Luego $f(2) = 12$ y $f'(2) = 14$, que coincide con $f'(x) = 3x^2 + 2$.

Observación 2.11 (Duales con varias componentes). Si se toman p infinitesimales $\varepsilon_1, \ldots, \varepsilon_p$ con $\varepsilon_r\varepsilon_s = 0$ para todo r, s, cada variable lleva un vector de p partes duales, y con las semillas $x_k + \sum_r (\dot{x}^{(r)})_k \varepsilon_r$ una sola pasada calcula p productos $J\dot{x}^{(r)}$. Con $p = n$ y semillas $e_1, \ldots, e_n$ se obtiene el jacobiano entero en una pasada vectorial, cuyo coste sigue siendo proporcional a $n \cdot \mathrm{ops}(f)$ [17]. La biblioteca JAX ofrece esta operación como `jvp` (*Jacobian-vector product*).

2.3 Coste del modo directo

Proposición 2.12. *Existe una constante ω_{dir}, que depende solo del conjunto Φ de operaciones elementales, tal que una pasada en modo directo cuesta a lo sumo $\omega_{\mathrm{dir}} \cdot \mathrm{ops}(f)$ operaciones.*

Demostración. Cada operación de la traza se sustituye por su extensión dual. La parte real cuesta lo mismo que la operación original; la parte dual requiere evaluar a lo sumo k derivadas parciales, de coste acotado por hipótesis (definición 1.1), y una combinación lineal de k términos. El cociente entre ambos costes está acotado por una constante que depende de k y de Φ, y sumando sobre las ℓ operaciones se obtiene la cota. Por ejemplo, el producto dual usa tres productos y una suma en lugar de un producto. ∎

Observación 2.13 (Conexión con los LLM). El modo directo no se usa para el gradiente de la pérdida de un LLM, pero sí aparece en otros cálculos: un producto hessiana-vector se obtiene aplicando el modo directo al gradiente (tema 5), y la memoria del modo directo no crece con la longitud de la traza, porque cada tangente se descarta en cuanto deja de usarse. El precio es el del corolario 2.4: una pasada por dirección.

Lecturas recomendadas para este tema

- Baydin *et al.* [2], §3.1 «Forward Mode» y §3.1.1 «Dual Numbers».
- Griewank y Walther [10], cap. 3 «Fundamentals of Forward and Reverse».
- MIT 18.337 [17], notas de la clase 8, apartados «Dual numbers», «Higher dimensions» y «Forward-Mode AD as jvp».
- MIT 18.S096 [18], clase 5, parte 2, «Forward Automatic Differentiation via Dual Numbers».

Ejercicios del tema 2

Ejercicio 2.1 ⋆ (a) Calcule la parte dual de $(3+\varepsilon)^2$ e interprétela. (b) Una función $f\colon \mathbb{R}^{1000} \to \mathbb{R}$ se diferencia en modo directo. ¿Cuántas pasadas hacen falta para obtener su gradiente? ¿Y para una sola derivada direccional?

Ejercicio 2.2 ⋆⋆ Con números duales, calcule $f(1)$ y $f'(1)$ para $f(x) = x^4 - 3x$.

Ejercicio 2.3 ⋆⋆ Con números duales y la proposición 2.6, calcule $g(2)$ y $g'(2)$ para $g(x) = x/(1+x^2)$.

Ejercicio 2.4 ⋆⋆ Con la extensión dual de tanh, calcule $h(1)$ y $h'(1)$ para $h(x) = \tanh(x^2)$ con cuatro cifras significativas, y compruébelo con la regla de la cadena.

Ejercicio 2.5 ⋆⋆ Para $f(x_1, x_2) = \ln x_1 + x_1 x_2 - \operatorname{sen} x_2$ en $(2, 5)$, evalúe la traza del ejemplo 1.5 con duales de dos componentes, $x_1 = 2 + \varepsilon_1$ y $x_2 = 5 + \varepsilon_2$ ($\varepsilon_r \varepsilon_s = 0$), y obtenga el gradiente en una sola pasada. Deduzca la derivada direccional en la dirección $(1, 1)^\top$.

Fuente: adaptado de MIT 18.337 [17], notas de la clase 8, apartado «Higher dimensions», aplicado al ejemplo de Baydin *et al.* [2, tabla 2].

Ejercicio 2.6 ⋆⋆⋆ Demuestre por inducción sobre el grado, usando solo la aritmética de la definición 2.5, que todo polinomio real p cumple $p(a + b\varepsilon) = p(a) + p'(a)\, b\, \varepsilon$. Demuestre después que, si f y g cumplen esa propiedad, también la cumple $g \circ f$.

Tema 3

El modo inverso y la retropropagación

El modo inverso recorre la traza dos veces: hacia delante, para calcular y guardar los valores, y hacia atrás, para propagar desde la salida la derivada de esta respecto a cada variable. Este tema define los adjuntos, demuestra que el algoritmo calcula un producto vector-jacobiano, estudia el papel del orden topológico y aplica el método a mano. Se siguen la §3.2 de [2], el capítulo 3 de [10], las §6.5.2–6.5.6 de [9], las notas «Backpropagation, Intuitions» de [5] y la §5.3 de [23].

3.1 Adjuntos

Se fija una traza de $f\colon \mathbb{R}^n \to \mathbb{R}$ con salida escalar $y = v_\ell$, y un punto x en el que se han calculado todos los valores v_i.

Definición 3.1 (Adjuntos). Los *adjuntos* de la traza se obtienen así: se inicializan $\bar{v}_\ell = 1$ y $\bar{v}_j = 0$ para $j < \ell$, y se recorren las operaciones en orden decreciente, $i = \ell, \ell-1, \dots, 1$, ejecutando para cada argumento $j \prec i$ la actualización

$$\bar{v}_j \;\mathrel{+}=\; \bar{v}_i \, \frac{\partial \varphi_i}{\partial v_j}.$$

El valor final de $\bar{v}_j$ se llama *adjunto* de v_j.

Equivalentemente, el valor final de cada adjunto es

$$\bar{v}_j = \sum_{i\,:\,j \prec i} \bar{v}_i \, \frac{\partial \varphi_i}{\partial v_j},$$

la suma sobre todas las operaciones que usan v_j. El símbolo $+=$ es esencial: si v_j alimenta a varias operaciones, su adjunto acumula una contribución de cada una.

Para enunciar qué calculan los adjuntos, sea $F_i(v_{1-n}, \dots, v_i)$ la salida que se obtiene al ejecutar las operaciones $i+1, \dots, \ell$ de la traza a partir de unos valores *arbitrarios* de las variables $v_{1-n}, \dots, v_i$, tratadas como independientes. Así, $F_\ell(v_{1-n}, \dots, v_\ell) = v_\ell$ y $F_0(x) = f(x)$, y $\partial F_i / \partial v_i$ mide la sensibilidad de la salida a una perturbación de v_i introducida justo después de calcularla.

Teorema 3.2 (Corrección del modo inverso). *Para cada $i \in \{0, \dots, \ell\}$, después de procesar las operaciones $\ell, \dots, i+1$, los acumuladores cumplen $\bar{v}_j = \partial F_i / \partial v_j$ para todo $j \le i$. En particular, al terminar,*

$$\bar{v}_{k-n} = \frac{\partial f}{\partial x_k}(x) \quad (k = 1, \dots, n), \qquad \textit{es decir,} \qquad (\bar{v}_{1-n}, \dots, \bar{v}_0) = \nabla f(x)^\top,$$

y el adjunto de cada variable intermedia es $\bar{v}_i = \partial F_i / \partial v_i$.

Demostración. Por inducción descendente sobre i. Para $i = \ell$ no se ha procesado ninguna operación: $\bar{v}_\ell = 1 = \partial F_\ell / \partial v_\ell$ y $\bar{v}_j = 0 = \partial F_\ell / \partial v_j$ para $j < \ell$, porque $F_\ell = v_\ell$.

Supóngase cierto para i y procésese la operación i. Por definición de las funciones F,

$$F_{i-1}(v_{1-n}, \dots, v_{i-1}) = F_i\big(v_{1-n}, \dots, v_{i-1}, \varphi_i((v_j)_{j \prec i})\big).$$

Por la regla de la cadena, para $j \le i - 1$,

$$\frac{\partial F_{i-1}}{\partial v_j} = \frac{\partial F_i}{\partial v_j} + \frac{\partial F_i}{\partial v_i}\,\frac{\partial \varphi_i}{\partial v_j},$$

donde $\partial \varphi_i / \partial v_j = 0$ si $j \not\prec i$. Por la hipótesis de inducción, el primer sumando es el valor actual de $\bar{v}_j$ y el factor $\partial F_i / \partial v_i$ es el valor actual de $\bar{v}_i$. La actualización de la definición 3.1 suma exactamente $\bar{v}_i\, \partial \varphi_i / \partial v_j$ a $\bar{v}_j$ para $j \prec i$ y no modifica los demás, luego tras ella $\bar{v}_j = \partial F_{i-1} / \partial v_j$ para todo $j \le i - 1$.

Para $i = 0$ se obtiene $\bar{v}_{k-n} = \partial F_0 / \partial x_k = \partial f / \partial x_k$. Por último, el acumulador $\bar{v}_i$ no cambia después de procesar la operación i, porque las operaciones $i - 1, \dots, 1$ solo actualizan sus argumentos, de índice menor que el suyo; su valor final es, pues, $\partial F_i / \partial v_i$. ■

La demostración muestra por qué el orden importa: cuando se procesa la operación i, el acumulador $\bar{v}_i$ debe estar completo, lo que exige haber procesado antes todas las operaciones que usan v_i. Es la regla de la cadena en un grafo del módulo 9 (tema 5), ahora con su algoritmo.

Proposición 3.3 (El modo inverso calcula $\bar{y}^\top J$). *Sea $f\colon \mathbb{R}^n \to \mathbb{R}^m$ y $\bar{y} \in \mathbb{R}^m$. Si se inicializan los adjuntos de las salidas con las componentes de $\bar{y}$ y se ejecuta el recorrido de la definición 3.1, los adjuntos de las entradas forman el vector fila $\bar{x}^\top = \bar{y}^\top J_f(x)$. En consecuencia, el jacobiano completo requiere m pasadas inversas, una por fila.*

Demostración. Se añade a la traza una operación final $v_{\ell+1} = \sum_r \bar{y}_r\, y_r$, lineal en las salidas, y se aplica el teorema 3.2 a la función escalar $g(x) = \bar{y}^\top f(x)$: los adjuntos de las entradas son $\nabla g(x)^\top = \bar{y}^\top J_f(x)$. Procesar la operación añadida asigna a cada salida y_r el adjunto $\bar{y}_r$, que es la inicialización del enunciado. Con $\bar{y} = e_r$ se obtiene la fila r de $J_f(x)$. ■

3.2 El algoritmo

Modo inverso (entrada: traza de $f\colon \mathbb{R}^n \to \mathbb{R}$, punto x).

1. *Pasada hacia delante.* Para $i = 1, \dots, \ell$: calcular $v_i = \varphi_i((v_j)_{j \prec i})$ y guardar los valores que necesitarán las derivadas locales.
2. *Inicialización.* $\bar{v}_\ell \leftarrow 1$; $\bar{v}_j \leftarrow 0$ para $j < \ell$.
3. *Pasada hacia atrás.* Para $i = \ell, \ell - 1, \dots, 1$ y para cada $j \prec i$: $\bar{v}_j \leftarrow \bar{v}_j + \bar{v}_i\, \partial \varphi_i / \partial v_j$.
4. *Salida.* $\nabla f(x) = (\bar{v}_{1-n}, \dots, \bar{v}_0)^\top$.

Figura 3.1: El modo inverso de la diferenciación automática [2, §3.2]. En redes neuronales se llama *retropropagación* [9, §6.5.6].

Ejemplo 3.4 (Traza adjunta del ejemplo de Baydin *et al.*). Para el ejemplo 1.5, la pasada hacia atrás es [2, tabla 3]:

Operación procesada	Actualizaciones	Valores
semilla	$\bar{v}_5 = 1$	
$v_5 = v_4 - v_3$	$\bar{v}_4 \mathrel{+}= \bar{v}_5$; $\bar{v}_3 \mathrel{+}= -\bar{v}_5$	$\bar{v}_4 = 1$; $\bar{v}_3 = -1$
$v_4 = v_1 + v_2$	$\bar{v}_1 \mathrel{+}= \bar{v}_4$; $\bar{v}_2 \mathrel{+}= \bar{v}_4$	$\bar{v}_1 = 1$; $\bar{v}_2 = 1$
$v_3 = \operatorname{sen} v_0$	$\bar{v}_0 \mathrel{+}= \bar{v}_3 \cos v_0$	$\bar{v}_0 = -0{,}2837$
$v_2 = v_{-1} v_0$	$\bar{v}_{-1} \mathrel{+}= \bar{v}_2 v_0$; $\bar{v}_0 \mathrel{+}= \bar{v}_2 v_{-1}$	$\bar{v}_{-1} = 5$; $\bar{v}_0 = 1{,}7163$
$v_1 = \ln v_{-1}$	$\bar{v}_{-1} \mathrel{+}= \bar{v}_1 / v_{-1}$	$\bar{v}_{-1} = 5{,}5$

Una sola pasada da las dos derivadas, $\partial f/\partial x_1 = 5{,}5$ y $\partial f/\partial x_2 = 1{,}7163$, que en el ejemplo 2.3 requerían dos pasadas en modo directo. Las variables v_{-1} y v_0 alimentan a dos operaciones cada una, y sus adjuntos acumulan dos contribuciones.

Ejemplo 3.5 (Una expresión con cuatro entradas). Sea $L = (a \cdot b + c) \cdot f$ con $a = 3$, $b = -2$, $c = 8$, $f = -1$, y variables intermedias $e = ab$ y $d = e + c$. Hacia delante: $e = -6$, $d = 2$, $L = -2$. Hacia atrás, con $\bar{L} = 1$:

1. nodo $L = d \cdot f$: $\bar{d} = \bar{L} \cdot f = -1$ y $\bar{f} = \bar{L} \cdot d = 2$;
2. nodo $d = e + c$: la suma copia el adjunto, $\bar{e} = \bar{c} = \bar{d} = -1$;
3. nodo $e = a \cdot b$: el producto intercambia los factores, $\bar{a} = \bar{e} \cdot b = 2$ y $\bar{b} = \bar{e} \cdot a = -3$.

En efecto, $L = -ab - 8$ y $\partial L/\partial a = -b = 2$. La figura 3.2 muestra el grafo con los dos recorridos.

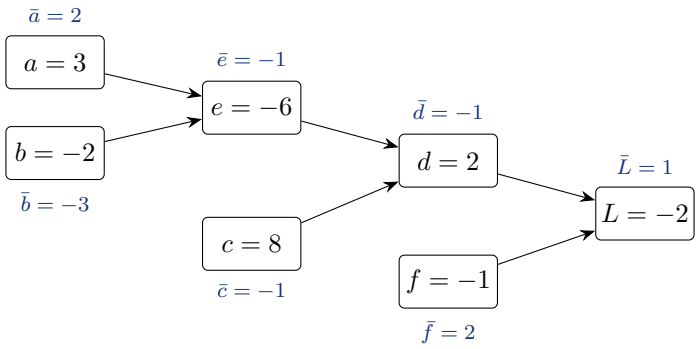


Figura 3.2: Grafo del ejemplo 3.5. Dentro de cada nodo, el valor de la pasada hacia delante; junto a él, en color, su adjunto, obtenido en la pasada hacia atrás.

Ejemplo 3.6 (Una neurona). Sea $o = \tanh(n)$ con $n = w_1 x_1 + w_2 x_2 + b$, $x = (1, -2)$, $w = (0{,}5;\ 0{,}25)$ y $b = 0{,}5$. Hacia delante, $n = 0{,}5 - 0{,}5 + 0{,}5 = 0{,}5$ y $o = \tanh 0{,}5 \approx 0{,}4621$. Hacia atrás, con $\bar{o} = 1$: el nodo tanh da $\bar{n} = \bar{o}(1 - o^2) = 0{,}7864$; la suma lo copia en $\bar{b} = 0{,}7864$ y en cada producto; los productos dan $\bar{w}_1 = \bar{n}\, x_1 = 0{,}7864$, $\bar{w}_2 = \bar{n}\, x_2 = -1{,}5729$, $\bar{x}_1 = \bar{n}\, w_1 = 0{,}3932$ y $\bar{x}_2 = \bar{n}\, w_2 = 0{,}1966$.

Las derivadas locales usan valores de la pasada hacia delante: o en el nodo tanh, los factores en cada producto. Por eso el modo inverso debe *guardar* esos valores hasta la pasada hacia atrás; en un LLM, las activaciones guardadas ocupan a menudo más memoria que los pesos (tema 5).

3.3 El orden topológico

Definición 3.7 (Orden topológico). Un *orden topológico* de un grafo dirigido $G = (V, E)$ es una enumeración $u_1, \ldots, u_{|V|}$ de sus nodos tal que toda arista $u_r \to u_s$ cumple $r < s$.

El teorema 3.2 se ha enunciado con el orden de la traza, pero la demostración solo usa que cada operación se procesa después de todas las que usan su resultado. Por tanto, el modo inverso es correcto si las operaciones se procesan en el *inverso* de cualquier orden topológico del grafo.

Proposición 3.8 (Construcción por búsqueda en profundidad). *Sea G un grafo dirigido acíclico y sea el procedimiento* VISITAR(v)*: si v no está marcado, marcarlo, llamar a* VISITAR(u) *para cada predecesor u de v (cada u con $u \to v$) y, después, añadir v al final de una lista. Si se llama a* VISITAR *desde la salida, la lista resultante es un orden topológico del subgrafo de los nodos de los que depende la salida, y se construye en tiempo $O(|V| + |E|)$.*

Demostración. Sea $u \to v$ una arista con v en la lista. Cuando se añade v, ya se ha ejecutado VISITAR(u). Si esa llamada marcó u por primera vez, terminó añadiendo u antes de volver. Si u ya estaba marcado, o bien ya estaba en la lista, o bien su llamada seguía en curso; en este último caso v se habría alcanzado desde u siguiendo predecesores, lo que daría un camino $v \to \cdots \to u$ que, junto con $u \to v$, formaría un ciclo, imposible en un grafo acíclico. En ambos casos u precede a v. Cada nodo se marca una vez y cada arista se examina una vez, de donde el coste. ■

Ejemplo 3.9 (Qué ocurre si el orden es incorrecto). Sea $x = 3$, $u = 2x$, $y = u^2$, $z = y+u$. Entonces $z = 4x^2 + 2x$ y $dz/dx = 8x + 2 = 26$.

- Orden inverso correcto (z, y, u): $\bar{z} = 1$; desde z, $\bar{y} = 1$ y $\bar{u} = 1$; desde y, $\bar{u}$ += $2u\,\bar{y} = 12$, luego $\bar{u} = 13$; desde u, $\bar{x} = 2 \cdot 13 = 26$.
- Orden incorrecto (z, u, y): u se procesa cuando $\bar{u} = 1$ y da $\bar{x} = 2$. Después, y suma 12 a $\bar{u}$, pero u ya no vuelve a propagar. Resultado: $\bar{x} = 2$, erróneo.

Observación 3.10 (La cinta). Muchos sistemas no construyen el orden con la proposición 3.8, sino que registran las operaciones en una lista, la *cinta* (*tape* o lista de Wengert), en el orden en que se ejecutan. Ese orden es topológico, porque ninguna operación se ejecuta antes que sus argumentos, y basta recorrer la cinta al revés. Es el diseño de la tarea complementaria de autograd del curso 11-785 de Carnegie Mellon [4, §2.3], que sustituye el grafo por una lista lineal de operaciones.

Observación 3.11 (Conexión con los LLM). En un Transformer, cada bloque suma su salida a la entrada mediante una conexión residual, $h_{k+1} = h_k + F_k(h_k)$. La variable h_k alimenta así a dos operaciones, F_k y la suma, como u en el ejemplo 3.9, y su adjunto es la suma de dos contribuciones: $\bar{h}_k = \bar{h}_{k+1} + \bar{h}_{k+1}\, J_{F_k}(h_k)$. El primer sumando transmite el gradiente sin atenuarse a través de toda la pila de bloques (módulos 11 y 14).

Lecturas recomendadas para este tema

- Baydin *et al.* [2], §3.2 «Reverse Mode», con la tabla 3.
- Griewank y Walther [10], cap. 3 «Fundamentals of Forward and Reverse».

- Goodfellow, Bengio y Courville [9], §6.5.2 «Chain Rule of Calculus», §6.5.3 «Recursively Applying the Chain Rule to Obtain Backprop» y §6.5.6 «General Back-Propagation».
- Stanford CS231n [5], «Backpropagation, Intuitions», apartados «Intuitive understanding of backpropagation» y «Patterns in backward flow».
- Zhang *et al.* [23], §5.3 «Forward Propagation, Backward Propagation, and Computational Graphs».

Ejercicios del tema 3

Ejercicio 3.1 ⋆ ¿Por qué la actualización de la definición 3.1 usa += y no una asignación? Dé un ejemplo de una sola operación en el que la asignación produzca un resultado erróneo.

Ejercicio 3.2 ⋆⋆ Dibuje el grafo de $f(x, y) = (x + y) \cdot (x \cdot y)$ y retropropague a mano en $(2, 3)$. Dé f, $\partial f/\partial x$ y $\partial f/\partial y$, y compruébelos derivando la expresión.

Ejercicio 3.3 ⋆⋆ Para la neurona $o = \tanh(w_1 x_1 + w_2 x_2 + b)$ con $x = (2, 0)$, $w = (-1, 3)$ y $b = 1$, calcule o y las derivadas de o respecto a w_1, w_2, b, x_1 y x_2.

Ejercicio 3.4 ⋆⋆ Para $f = a \cdot b + a$ con $a = 2$ y $b = 5$, escriba todos los órdenes topológicos del grafo, retropropague y dé $\partial f/\partial a$ y $\partial f/\partial b$.

Ejercicio 3.5 ⋆⋆ Sobre el grafo construido en el ejercicio 1.3, para $f(x) = \ln(x^2) \cdot \operatorname{sen} x + x^{-1}$: (a) coloque en cada arista la derivada local correspondiente; (b) calcule $f'(2)$ recorriendo el grafo desde x hasta f (modo directo) y (c) desde f hasta x (modo inverso). Compruebe que coinciden con la derivada simbólica.

Fuente: adaptado de Zhang *et al.* [23], §2.5 «Automatic Differentiation», ejercicios 6 y 7.

Ejercicio 3.6 ⋆⋆ Se añade un sesgo b_1, que no se regulariza, a la capa oculta de un perceptrón con decaimiento de pesos. Con vectores fila: $z = xW_1 + b_1$, $h = \max(0, z)$, $o = hW_2$, $L = \frac{1}{2}(o - y)^2$, $s = \frac{\lambda}{2}\big(\|W_1\|_F^2 + \|W_2\|_F^2\big)$ y $J = L + s$. Sean $x = (1, 2)$, $W_1 = \left(\begin{smallmatrix} 1 & 0 \\ -1 & 0{,}5 \end{smallmatrix}\right)$, $b_1 = (0{,}5;\ 0)$, $W_2 = \left(\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right)$, $y = 3$ y $\lambda = 0{,}1$. (a) Escriba las ecuaciones de la pasada hacia atrás del grafo. (b) Calcule J, $\nabla_{W_2} J$, $\nabla_{W_1} J$ y $\nabla_{b_1} J$.

Fuente: adaptado de Zhang *et al.* [23], §5.3, ejercicio 2 (se añaden datos numéricos).

Ejercicio 3.7 ⋆⋆⋆ Se modifica el algoritmo de la figura 3.1 sustituyendo la acumulación por la asignación $\bar{v}_j \leftarrow \bar{v}_i \, \partial \varphi_i / \partial v_j$. Demuestre que el resultado sigue siendo correcto si cada variable es argumento de a lo sumo una operación (el grafo es entonces un árbol cuya raíz es la salida), y que puede ser incorrecto en caso contrario, con un ejemplo.

Tema 4

Coste del gradiente y productos vector-jacobiano

Este tema responde a la pregunta que justifica el módulo: por qué el modo inverso calcula el gradiente de una función de millones de variables con un coste comparable al de evaluarla. Se demuestra el principio del gradiente barato, se compara el coste de los dos modos mediante la asociatividad del producto de jacobianos, se formulan las reglas locales de los sistemas reales como productos vector-jacobiano y se deduce de ellas la regla $C \approx 6ND$ del cómputo de entrenamiento. Se siguen el capítulo 4 de [10], la §3.2 de [2], la lección 6 de [12], el apartado «Gradients for vectorized operations» de [5] y el artículo de Kaplan *et al.* [13].

4.1 El principio del gradiente barato

Teorema 4.1 (Gradiente barato). *Sea $f\colon \mathbb{R}^n \to \mathbb{R}$ dada por una traza con operaciones del conjunto Φ de la definición 1.1, cada una con a lo sumo k argumentos. Existe una constante ω, que depende solo de Φ y no de n ni de la traza, tal que el modo inverso calcula $\nabla f(x)$ con a lo sumo $\omega \cdot \operatorname{ops}(f)$ operaciones.*

Demostración. Sea ℓ el número de operaciones de la traza y E el de aristas del grafo. Cada operación cuesta al menos una unidad, luego $\operatorname{ops}(f) \geq \ell$, y tiene a lo sumo k argumentos, luego $E \leq k\ell$.

1. La pasada hacia delante cuesta $\operatorname{ops}(f)$.
2. En la pasada hacia atrás, la operación i, con $r_i \leq k$ argumentos, evalúa r_i derivadas locales sobre valores guardados, cada una de coste a lo sumo c_Φ (definición 1.1), y ejecuta r_i productos y r_i sumas. Su coste es a lo sumo $(c_\Phi + 2)\, r_i$.
3. Sumando sobre las operaciones, la pasada hacia atrás cuesta a lo sumo $(c_\Phi + 2) \sum_i r_i = (c_\Phi + 2)E \leq (c_\Phi + 2)k\ell \leq (c_\Phi + 2)k \operatorname{ops}(f)$.

El total es a lo sumo $\big(1 + (c_\Phi + 2)k\big) \operatorname{ops}(f)$, y basta tomar $\omega = 1 + (c_\Phi + 2)k$. La corrección del resultado es el teorema 3.2. ∎

La constante de la demostración es pesimista. Un análisis más fino de cada operación da cotas pequeñas: Baydin *et al.*, siguiendo a Griewank y Walther, indican que el cociente entre el coste del modo inverso y el de la función está por debajo de 6 y suele estar entre 2 y 3 [2, §3.2], [10, cap. 4]. Para funciones racionales, un resultado de este tipo fue demostrado por Baur y Strassen [1]. Lo esencial es que la cota no depende de n: diez derivadas y 10^{11} derivadas cuestan, en relación con f, lo mismo.

Corolario 4.2 (Elección del modo). *Para $f\colon \mathbb{R}^n \to \mathbb{R}^m$, el jacobiano completo cuesta del orden de $n \cdot \mathrm{ops}(f)$ en modo directo y de $m \cdot \mathrm{ops}(f)$ en modo inverso. Conviene el directo si $n \ll m$ y el inverso si $m \ll n$. Entrenar un LLM corresponde a $n \approx 10^{11}$ y $m = 1$.*

Demostración. Es la combinación del corolario 2.4 y la proposición 2.12 con la proposición 3.3 y el teorema 4.1, aplicado este a cada una de las m funciones escalares $\bar{y}^\top f$ con $\bar{y} = e_r$. ■

El precio del modo inverso es la memoria: la pasada hacia atrás necesita los valores de la pasada hacia delante. En el modo directo, en cambio, cada tangente se descarta en cuanto deja de usarse. El tema 5 estudia este compromiso.

4.2 Asociatividad del producto de jacobianos

Sea $f = f_L \circ \cdots \circ f_1$ una composición de funciones diferenciables. Por la regla de la cadena (módulo 4), $J_f = J_{f_L} \cdots J_{f_1}$, con cada jacobiano evaluado en el punto correspondiente. El producto de matrices es asociativo y el resultado no depende del orden en que se agrupen los factores, pero el coste sí.

Proposición 4.3 (Coste de las dos agrupaciones). *Sea $f\colon \mathbb{R}^m \to \mathbb{R}$ con $J_f = J_L J_{L-1} \cdots J_1$, donde $J_L \in \mathbb{R}^{1\times m}$ y $J_1, \ldots, J_{L-1} \in \mathbb{R}^{m\times m}$, con $L \geq 2$, y cuéntese como coste de un producto $(p \times q)(q \times r)$ el número pqr de multiplicaciones escalares. Entonces:*

(i) agrupando de derecha a izquierda, $J_L\big(J_{L-1}(\cdots(J_2 J_1))\big)$, el coste es $(L-2)m^3 + m^2$;

(ii) agrupando de izquierda a derecha, $\big(((J_L J_{L-1})J_{L-2})\cdots\big)J_1$, el coste es $(L-1)m^2$.

Demostración. (i) Los $L-2$ productos J_2J_1, $J_3(J_2J_1)$, …, $J_{L-1}(\cdots)$ son de matrices $m \times m$ y cuestan m^3 cada uno; el último, J_L por una matriz $m \times m$, cuesta $1 \cdot m \cdot m$. (ii) Cada producto parcial es un vector fila $1 \times m$, y cada uno de los $L-1$ productos por una matriz $m \times m$ cuesta m^2. ■

La agrupación (i) es la del modo directo con todas las semillas a la vez: propaga la matriz $\partial v/\partial x$, de tamaño $m \times m$. La agrupación (ii) es la del modo inverso: lo que viaja hacia atrás es siempre un vector fila, el adjunto. Con $L = 3$ y $m = 100$, el cociente de costes es $(100^3 + 100^2)/(2 \cdot 100^2) = 50{,}5$, y en general crece linealmente con m. Para composiciones que no son cadenas, encontrar la agrupación óptima es un problema difícil, estudiado en [10, cap. 9 y 10].

4.3 Productos vector-jacobiano

Los sistemas reales no trabajan con escalares, sino con operaciones sobre tensores, y nunca construyen sus jacobianos [12].

Definición 4.4 (Producto vector-jacobiano). Sea $g\colon \mathbb{R}^p \to \mathbb{R}^q$ una operación con entrada u y salida $w = g(u)$, y sea $\bar{w} \in \mathbb{R}^{1\times q}$ el adjunto de la salida. El *producto vector-jacobiano* (VJP) de g en u es la aplicación lineal $\bar{w} \mapsto \bar{u} = \bar{w}\, J_g(u) \in \mathbb{R}^{1\times p}$.

Por el teorema 3.2, aplicado por bloques, el modo inverso sobre un grafo de operaciones tensoriales solo necesita, para cada tipo de operación, una función que calcule su VJP. Si la

entrada o la salida son matrices, se aplanan para la definición y el resultado se devuelve con la forma original.

Ejemplo 4.5 (Operaciones componente a componente). Si $w = \phi(u)$ se aplica componente a componente, $J_g(u) = \operatorname{diag}(\phi'(u))$ y el VJP es $\bar{u} = \bar{w} \odot \phi'(u)$, con coste $O(p)$ en lugar de los $O(p^2)$ que exigiría multiplicar por la matriz diagonal [12]. Para $\phi = \exp$, $\bar{u} = \bar{w} \odot w$: basta haber guardado la salida.

Proposición 4.6 (VJP de la capa lineal). *Sea $Y = XW$ con $X \in \mathbb{R}^{B\times n}$, $W \in \mathbb{R}^{n\times m}$, y sea $\bar{Y} \in \mathbb{R}^{B\times m}$ el adjunto de la salida. Entonces*

$$\bar{X} = \bar{Y}\, W^\top \in \mathbb{R}^{B\times n}, \qquad \bar{W} = X^\top \bar{Y} \in \mathbb{R}^{n\times m}.$$

Demostración. Se tiene $Y_{rs} = \sum_i X_{ri} W_{is}$, luego $\partial Y_{rs}/\partial X_{ri} = W_{is}$ y $\partial Y_{rs}/\partial W_{is} = X_{ri}$. Por la regla de la cadena, $\bar{X}_{ri} = \sum_s \bar{Y}_{rs} W_{is} = (\bar{Y}W^\top)_{ri}$ y $\bar{W}_{is} = \sum_r \bar{Y}_{rs} X_{ri} = (X^\top \bar{Y})_{is}$. Es el resultado obtenido en el módulo 4 (tema 3) y en el módulo 9 (tema 5), ahora como regla local de un sistema de AD. ■

Ejemplo 4.7. Sean $X = (1, 2)$, $W = \left(\begin{smallmatrix} 1 & 0 \\ -1 & 1 \end{smallmatrix}\right)$ y $\bar{Y} = (1, -1)$. Entonces $Y = (-1, 2)$, $\bar{X} = \bar{Y}W^\top = (1, -2)$ y $\bar{W} = X^\top \bar{Y} = \left(\begin{smallmatrix} 1 & -1 \\ 2 & -2 \end{smallmatrix}\right)$. Ambos gradientes tienen la forma de la variable a la que corresponden.

Proposición 4.8 (VJP de la softmax). *Sea $p = \operatorname{softmax}(z)$ con $z \in \mathbb{R}^{1\times K}$ y sea $\bar{p} \in \mathbb{R}^{1\times K}$. Entonces*

$$\bar{z} = \bar{p}\, J_{\operatorname{softmax}}(z) = p \odot \big(\bar{p} - \langle \bar{p}, p\rangle\, \mathbf{1}\big),$$

donde $\langle \bar{p}, p\rangle = \sum_k \bar{p}_k p_k$ y $\mathbf{1} = (1, \dots, 1)$. El cálculo cuesta $O(K)$ operaciones.

Demostración. Como se demostró en el módulo 4 (tema 3), $\partial p_i/\partial z_j = p_i(\mathbb{1}[i = j] - p_j)$. Luego $\bar{z}_j = \sum_i \bar{p}_i\, p_i(\mathbb{1}[i = j] - p_j) = \bar{p}_j p_j - p_j \sum_i \bar{p}_i p_i = p_j\big(\bar{p}_j - \langle \bar{p}, p\rangle\big)$. El producto escalar cuesta $O(K)$ y se calcula una vez; cada componente cuesta después $O(1)$. ■

Corolario 4.9 (Softmax con entropía cruzada). *Si $L = -\ln p_y$, entonces $\bar{z} = p - e_y$.*

Demostración. El VJP de $p \mapsto -\ln p_y$ con $\bar{L} = 1$ es $\bar{p} = -e_y/p_y$, y $\langle \bar{p}, p\rangle = -1$. Por la proposición 4.8, $\bar{z}_j = p_j(\bar{p}_j + 1)$, que vale $p_y(-1/p_y + 1) = p_y - 1$ si $j = y$ y p_j si $j \neq y$. ■

En la práctica, la softmax y la entropía cruzada se tratan como una sola operación con VJP $p - e_y$, que además es numéricamente estable: es lo que hace `F.cross_entropy` en PyTorch, que recibe logits y no probabilidades (tema 6).

4.4 El cómputo de un paso de entrenamiento

Se cuenta como FLOP una suma o un producto en coma flotante. Un producto de matrices $(p \times q)(q \times r)$ cuesta $pr(2q - 1) \approx 2pqr$ FLOPs.

Proposición 4.10 (Coste de una capa lineal). *Para $Y = XW$ con $X \in \mathbb{R}^{B\times n}$ y $W \in \mathbb{R}^{n\times m}$, la pasada hacia delante cuesta unos $2Bnm$ FLOPs y la pasada hacia atrás unos $4Bnm$: $2Bnm$ para $\bar{X}$ y $2Bnm$ para $\bar{W}$. El total es $6Bnm = 6 \cdot$ (parámetros) $\cdot$ (filas del lote).*

Demostración. XW es $(B \times n)(n \times m)$; $\bar{Y}W^\top$ es $(B \times m)(m \times n)$; $X^\top \bar{Y}$ es $(n \times B)(B \times m)$. Los tres productos tienen el mismo número de multiplicaciones, Bnm, y cuestan unos $2Bnm$ FLOPs cada uno. La capa tiene nm pesos. ■

Corolario 4.11 (Regla $6ND$). *Si el cómputo de un modelo con N parámetros está dominado por productos de matrices en los que cada parámetro actúa una vez por token, entrenarlo con D tokens cuesta $C \approx 6ND$ FLOPs: $2ND$ en las pasadas hacia delante y $4ND$ en las pasadas hacia atrás.*

Demostración. Se aplica la proposición 4.10 a cada capa, con B igual al número de tokens procesados, y se suma sobre las capas y sobre los pasos. ■

La aproximación ignora las operaciones que no son productos por pesos, como el producto de la atención entre consultas y claves, que crece con la longitud del contexto, y el hecho de que la primera capa no necesita $\bar{X}$. Kaplan *et al.* la usan como base de las leyes de escalado [13], que se estudian en el módulo 16. Por ejemplo, un modelo de $N = 10^8$ parámetros entrenado con $D = 2 \cdot 10^9$ tokens requiere $C \approx 6 \cdot 10^8 \cdot 2 \cdot 10^9 = 1{,}2 \cdot 10^{18}$ FLOPs; a 10^{14} FLOP/s efectivos, unos $1{,}2 \cdot 10^4$ s, algo más de tres horas.

Lecturas recomendadas para este tema

- Griewank y Walther [10], cap. 4 «Memory Issues and Complexity Bounds».
- Baydin *et al.* [2], §3.2, discusión final sobre el coste de los dos modos.
- Grosse [12], lección 6, apartado sobre productos vector-jacobiano.
- Stanford CS231n [5], «Backpropagation, Intuitions», apartado «Gradients for vectorized operations».
- Kaplan *et al.* [13], recuento de parámetros y de cómputo de un Transformer y aproximación $C \approx 6ND$.

Ejercicios del tema 4

Ejercicio 4.1 ⋆ Se dispone de $f\colon \mathbb{R}^{1000} \to \mathbb{R}$ y de $g\colon \mathbb{R} \to \mathbb{R}^{1000}$. ¿Qué modo de AD usaría para el jacobiano de cada una y cuántas pasadas necesitaría? ¿Cuántas necesitaría con el otro modo?

Ejercicio 4.2 ⋆ ¿Cuándo conviene el modo directo y cuándo el inverso? Responda atendiendo a tres criterios: la cantidad de datos intermedios que hay que guardar, la posibilidad de ejecutar pasos en paralelo y el tamaño de las matrices y vectores que intervienen.

Fuente: adaptado de Zhang *et al.* [23], §2.5 «Automatic Differentiation», ejercicio 8.

Ejercicio 4.3 ⋆⋆ Sea $\nabla f^\top = c\,B\,A$ con $c \in \mathbb{R}^{1\times 100}$ y $A, B \in \mathbb{R}^{100\times 100}$. Cuente las multiplicaciones escalares de las dos agrupaciones posibles y su cociente. Generalice a $c \in \mathbb{R}^{1\times m}$ y $A, B \in \mathbb{R}^{m\times m}$.

Ejercicio 4.4 ⋆⋆ Para la capa $Y = XW$ con $X = (2, -1)$, $W = \left(\begin{smallmatrix} 1 & 2 & 0 \\ 0 & 1 & -1 \end{smallmatrix}\right)$ y adjunto de la salida $\bar{Y} = (1, 0, 2)$, calcule Y, $\bar{X}$ y $\bar{W}$, y compruebe las formas. Verifique una componente de $\bar{W}$ derivando $L = \bar{Y}\,Y^\top$ directamente.

Ejercicio 4.5 ⋆⋆ Un modelo de $N = 10^9$ parámetros se entrena durante 10^5 pasos con lotes de 512 secuencias de 1024 tokens en una GPU de $495 \cdot 10^{12}$ FLOP/s de pico, con una utilización del 50 %. Suponiendo que la pasada hacia delante cuesta $2N$ FLOPs por token y la hacia atrás el doble que la de delante, estime los FLOPs totales y el tiempo de entrenamiento en horas y en días.

Fuente: adaptado de Stanford CS336, primavera de 2026 [6], tarea 1, problema `adamw_accounting`, apartado (d).

Ejercicio 4.6 ⋆⋆⋆ (a) Demuestre la proposición 4.8 sin usar el jacobiano de la softmax, derivando $\bar{z}_j = \sum_i \bar{p}_i \, \partial p_i / \partial z_j$ a partir de $\ln p_i = z_i - \ln \sum_k e^{z_k}$. (b) Deduzca que, para $L = -\ln \mathrm{softmax}(z)_y$, $\partial L / \partial z_i = \mathrm{softmax}(z)_i - \mathbb{1}[i = y]$, y que las componentes de $\nabla_z L$ suman 0. (c) Compruébelo con $z = (2, 1, 0)$ e $y = 0$, y dé el valor de L.

Tema 5

Memoria, segundo orden y método adjunto

El modo inverso compra su eficiencia en operaciones con memoria: necesita los valores de la pasada hacia delante. Este tema cuantifica esa memoria y estudia el recálculo por puntos de control, que la reduce a cambio de cómputo; obtiene productos hessiana-vector combinando los dos modos, y muestra que los adjuntos son multiplicadores de Lagrange, lo que conecta la retropropagación con el método adjunto del control óptimo y de la física. Se siguen los capítulos 4 y 12 de [10], el artículo de Chen *et al.* [3], el de Pearlmutter [20], la §3.3 de [2] y la tarea 2 de CS336 [7].

5.1 La memoria del modo inverso

Proposición 5.1 (Memoria de una cadena de capas). *Sea $f = f_L \circ \cdots \circ f_1$ una red de L capas cuyas activaciones intermedias $h_k = f_k(h_{k-1})$ ocupan a unidades de memoria cada una, y supóngase que el VJP de la capa k necesita h_{k-1}. Evaluar f requiere memoria $O(a)$, mientras que el modo inverso de la figura 3.1 requiere guardar $h_0, \ldots, h_{L-1}$, es decir, memoria La.*

Demostración. Para evaluar f basta conservar la activación actual: h_k se calcula a partir de h_{k-1}, que puede descartarse a continuación. En el modo inverso, el VJP de la capa L se ejecuta en primer lugar y necesita h_{L-1}, y el de la capa 1, que se ejecuta en último lugar, necesita h_0. Como todas las pasadas hacia atrás ocurren después de la pasada hacia delante completa, las L activaciones deben estar disponibles a la vez. ■

En un LLM, a es proporcional al número de tokens del lote por la dimensión del modelo, y la memoria de las activaciones supera a menudo la de los pesos (módulo 17).

5.2 Recálculo por puntos de control

La idea del recálculo (*gradient checkpointing* o *rematerialization*) consiste en guardar solo algunas activaciones, los *puntos de control*, y recalcular las demás cuando la pasada hacia atrás las necesita [3], [10, cap. 12].

Proposición 5.2 (Un nivel de puntos de control). *En la situación de la proposición 5.1, divídanse las L capas en L/k tramos de k capas consecutivas y guárdese durante la pasada hacia delante solo la entrada de cada tramo. En la pasada hacia atrás, al llegar a cada tramo se repite su pasada hacia delante desde el punto de control, guardando sus k activaciones, y se retropropaga por él. Entonces:*

(i) la memoria de activaciones es $M(k) = (L/k + k)\,a$*, mínima para* $k = \sqrt{L}$*, con valor* $2\sqrt{L}\,a$*;*

(ii) el cómputo adicional es una pasada hacia delante más.

Demostración. (i) En todo momento se mantienen los L/k puntos de control y, como mucho, las activaciones de un tramo, k en total. La función $M(k)/a = L/k + k$, con $k > 0$ real, tiene derivada $-L/k^2 + 1$, que se anula en $k = \sqrt{L}$, y segunda derivada $2L/k^3 > 0$: es un mínimo, de valor $\sqrt{L} + \sqrt{L} = 2\sqrt{L}$. (ii) Cada capa se evalúa dos veces hacia delante, una en la pasada original y otra al recalcular su tramo. ■

Con las cuentas de la proposición 4.10, una pasada hacia delante extra cuesta $2N$ FLOPs por token, y el paso de entrenamiento pasa de $6N$ a $8N$: un 33 % más de cómputo a cambio de reducir la memoria de L a $2\sqrt{L}$ activaciones. Aplicando la idea de forma recursiva, la memoria baja a $O(\log L)$ con un cómputo de $O(L \log L)$ evaluaciones de capa [3]; Griewank y Walther estudian los esquemas óptimos de este tipo [10, cap. 12].

Observación 5.3 (Conexión con los LLM)*.* El recálculo de activaciones por bloques del Transformer es una técnica habitual en el entrenamiento de modelos grandes: los puntos de control son las entradas de los bloques y lo que se recalcula son las activaciones internas de la atención y de la red prealimentada. La tarea 2 de CS336 [7] pide analizar exactamente este compromiso (ejercicio 5.5).

5.3 Productos hessiana-vector

Las derivadas de segundo orden se obtienen aplicando la AD a un programa que, a su vez, calcula derivadas.

Proposición 5.4 (Directo sobre inverso). *Sea* $f\colon \mathbb{R}^n \to \mathbb{R}$ *de clase* C^2 *dada por una traza cuyas operaciones elementales son de clase* C^2*, y sean* $x, v \in \mathbb{R}^n$*. Si se ejecuta el modo inverso (que calcula* ∇f*) en aritmética dual, con entradas* $x + v\varepsilon$*, la parte dual del resultado es el producto hessiana-vector* $H_f(x)\,v$*. Su coste es a lo sumo una constante, independiente de* n*, por* $\operatorname{ops}(f)$*.*

Demostración. El modo inverso es un programa g, compuesto de operaciones elementales y de sus derivadas, que calcula $g(x) = \nabla f(x)$. Por el teorema 2.9, aplicado a g, la parte dual de $g(x + v\varepsilon)$ es $J_g(x)\,v$, y $J_g = J_{\nabla f} = H_f$. Por el teorema 4.1, $\operatorname{ops}(g) \le \omega \operatorname{ops}(f)$, y por la proposición 2.12 la pasada dual cuesta a lo sumo $\omega_{\text{dir}} \operatorname{ops}(g) \le \omega_{\text{dir}}\,\omega \operatorname{ops}(f)$. ■

Ejemplo 5.5. Sea $f(x_1, x_2) = x_1^2 x_2$ en $(1, 2)$ con $v = (1, 0)^\top$. El modo inverso calcula $\nabla f = (2x_1x_2,\, x_1^2)$. Con $x_1 = 1 + \varepsilon$ y $x_2 = 2$, se obtiene $\big(2(1+\varepsilon)\cdot 2,\, (1+\varepsilon)^2\big) = (4 + 4\varepsilon,\, 1 + 2\varepsilon)$. Luego $\nabla f(1, 2) = (4, 1)$ y $Hv = (4, 2)$, que es la primera columna de $H = \left(\begin{smallmatrix} 2x_2 & 2x_1 \\ 2x_1 & 0 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 4 & 2 \\ 2 & 0 \end{smallmatrix}\right)$.

La técnica es la de Pearlmutter [20] y permite usar información de curvatura sin formar el hessiano, que para $n = 10^9$ tendría 10^{18} entradas. El hessiano completo requiere n productos hessiana-vector, uno por columna, y cuesta del orden de $n \cdot \operatorname{ops}(f)$, frente al $O(\operatorname{ops}(f))$ del gradiente.

5.4 Los adjuntos como multiplicadores de Lagrange

Teorema 5.6 (Método adjunto). *Sea una traza de $f\colon \mathbb{R}^n \to \mathbb{R}$ y considérese la lagrangiana*

$$\mathcal{L}(x, v_1, \dots, v_\ell, \lambda_1, \dots, \lambda_\ell) = v_\ell - \sum_{i=1}^{\ell} \lambda_i \Big(v_i - \varphi_i\big((v_j)_{j \prec i}\big) \Big),$$

donde $v_{k-n} = x_k$. Si los v_i toman los valores de la traza y los multiplicadores se eligen de modo que $\partial \mathcal{L}/\partial v_i = 0$ para $i = 1, \dots, \ell$, entonces $\lambda_i = \bar{v}_i$ para todo i y $\nabla f(x) = \nabla_x \mathcal{L}$.

Demostración. Para $i \geq 1$, el término v_i aparece con coeficiente $-\lambda_i$ en su propia restricción, con coeficiente 1 en v_ℓ si $i = \ell$, y dentro de φ_r para cada r con $i \prec r$. Por tanto,

$$\frac{\partial \mathcal{L}}{\partial v_i} = \mathbb{1}[i = \ell] - \lambda_i + \sum_{r \,:\, i \prec r} \lambda_r \, \frac{\partial \varphi_r}{\partial v_i}.$$

Anularla da $\lambda_\ell = 1$ (la operación ℓ no es argumento de ninguna otra) y $\lambda_i = \sum_{r : i \prec r} \lambda_r \, \partial \varphi_r / \partial v_i$ para $i < \ell$, que es la recurrencia de la definición 3.1, resuelta en orden decreciente. Luego $\lambda_i = \bar{v}_i$.

Sobre los valores de la traza todas las restricciones se cumplen, y $\mathcal{L} = v_\ell = f(x)$ para todo x y toda elección de λ. Derivando respecto a x_k esta identidad, con $v_i = v_i(x)$, se obtiene $\partial f / \partial x_k = \partial \mathcal{L}/\partial x_k + \sum_{i \geq 1} (\partial \mathcal{L}/\partial v_i)(\partial v_i / \partial x_k) = \partial \mathcal{L}/\partial x_k$, porque los multiplicadores anulan cada $\partial \mathcal{L}/\partial v_i$. Finalmente, $\partial \mathcal{L}/\partial x_k = \sum_{r : k-n \prec r} \lambda_r \partial \varphi_r / \partial v_{k-n} = \bar{v}_{k-n}$. ∎

Ejemplo 5.7 (Una recurrencia en el tiempo). Sea $x_{t+1} = a\, x_t$ para $t = 0, 1, 2$, con $x_0 = 1$ y $a = 2$, y sea $L = \tfrac{1}{2} x_3^2$. Hacia delante, $x = (1, 2, 4, 8)$ y $L = 32$. Los multiplicadores cumplen $\lambda_3 = \partial L / \partial x_3 = x_3 = 8$ y $\lambda_t = \lambda_{t+1} \, \partial x_{t+1} / \partial x_t = a \lambda_{t+1}$: $\lambda_2 = 16$, $\lambda_1 = 32$. El parámetro a interviene en las tres transiciones, y $dL/da = \sum_t \lambda_{t+1} \, \partial x_{t+1} / \partial a = \lambda_3 x_2 + \lambda_2 x_1 + \lambda_1 x_0 = 32 + 32 + 32 = 96$. En efecto, $L = \tfrac{1}{2} a^6$ y $dL/da = 3a^5 = 96$.

El ejemplo es la retropropagación a través del tiempo de una red recurrente: un mismo parámetro actúa en cada paso y su gradiente es la suma de las contribuciones de todos ellos. En control óptimo, en diseño aerodinámico y en la asimilación de datos meteorológicos (el método 4D-Var), el mismo esquema se aplica a ecuaciones diferenciales, integrando una ecuación adjunta hacia atrás en el tiempo para obtener el gradiente de un coste respecto a millones de condiciones iniciales o parámetros.

Observación 5.8 (Historia). Linnainmaa describió el modo inverso en 1970 como herramienta para analizar errores de redondeo acumulados [15]; Rumelhart, Hinton y Williams lo popularizaron en 1986 para entrenar redes neuronales [22]. Baydin *et al.* reconstruyen esta historia y sus redescubrimientos [2, §3.3].

Lecturas recomendadas para este tema

- Griewank y Walther [10], cap. 4 «Memory Issues and Complexity Bounds» y cap. 12 «Reversal Schedules and Checkpointing».
- Chen *et al.* [3], «Training Deep Nets with Sublinear Memory Cost».
- Pearlmutter [20], «Fast exact multiplication by the Hessian».
- Baydin *et al.* [2], §3.3 «Origins of AD and Backpropagation».
- Stanford CS336 [7], tarea 2, sección 3.2.1 sobre recálculo de activaciones.

Ejercicios del tema 5

Ejercicio 5.1 ⋆ ¿Por qué la segunda derivada es mucho más costosa de calcular que la primera? Distinga entre un producto hessiana-vector y el hessiano completo de $f\colon \mathbb{R}^n \to \mathbb{R}$.

Fuente: adaptado de Zhang *et al.* [23], §2.5 «Automatic Differentiation», ejercicio 1.

Ejercicio 5.2 ⋆⋆ Sea $f(x_1, x_2) = x_1^2 x_2 + x_2^3$. Calcule $\nabla f(1,2)$ y el producto $H_f(1,2)\, v$ con $v = (0,1)^\top$ aplicando aritmética dual a la expresión del gradiente, como en el ejemplo 5.5. Compruébelo con el hessiano.

Ejercicio 5.3 ⋆⋆ Sea la recurrencia $x_{t+1} = a\, x_t + b$ para $t = 0, 1, 2$, con $x_0 = 1$, $a = 2$ y $b = 1$, y sea $L = x_3$. Calcule la trayectoria, los multiplicadores λ_t del teorema 5.6 y las derivadas dL/da y dL/db. Compruébelas con la expresión cerrada de x_3.

Ejercicio 5.4 ⋆⋆⋆ Demuestre la proposición 5.2 y calcule la memoria óptima para $L = 16$ y $L = 100$, comparándola con la del modo inverso sin puntos de control. ¿Cuántos FLOPs por parámetro y por token cuesta entonces un paso de entrenamiento?

Ejercicio 5.5 ⋆⋆⋆ Un Transformer tiene $N_b = 16$ bloques idénticos. Se mide la memoria en «bloques»: guardar las activaciones internas de un bloque cuesta 1 y guardar un punto de control cuesta $c = 1/4$. Con un nivel de recálculo en tramos de k bloques, la memoria pico es $M(k) = (N_b/k)\, c + k$. (a) Calcule $M(k)$ para $k \in \{1, 2, 4, 8, 16\}$ y el óptimo continuo. (b) Si se permite anidar los puntos de control sin límite de cómputo, describa una estrategia que reduzca la memoria a $O(\log N_b)$ y estime su cómputo.

Fuente: adaptado de Stanford CS336, primavera de 2026 [7], tarea 2, sección 3.2.1, problema `gradient_checkpointing`, apartados (a) y (b).

Tema 6

Implementación: un autograd escalar y PyTorch

Todo lo anterior cabe en unas decenas de líneas de Python. Este tema describe un sistema de diferenciación automática en modo inverso que opera con escalares, como micrograd [14], demuestra que basta un puñado de reglas locales, explica por qué los sistemas reales trabajan con tensores y presenta las piezas de PyTorch que se usan en el resto del curso: tensores con `requires_grad`, `backward`, `nn.Module` y los optimizadores. Se siguen el artículo de Paszke *et al.* [19], la nota «Autograd mechanics» de la documentación de PyTorch [21], la §2.5 de [23] y la tarea complementaria de autograd del curso 11-785 de Carnegie Mellon [4].

6.1 Un autograd escalar

Definición 6.1 (Nodo de un autograd escalar). Un *nodo* es un objeto con cuatro atributos: su valor `dato`; su adjunto `grad`, inicializado a 0; la lista `_prev` de los nodos de los que se ha calculado, y una función `_backward` que, leída la variable `grad` del propio nodo, suma a la `grad` de cada nodo de `_prev` su contribución según la regla local. Cada operación entre nodos crea un nodo nuevo y le asigna su `_backward`.

Operación	Hacia delante	Regla local (`_backward`)
$s = a + b$	$a + b$	$\bar{a}$ += $\bar{s}$; $\bar{b}$ += $\bar{s}$
$p = a \cdot b$	$a \cdot b$	$\bar{a}$ += $b\,\bar{p}$; $\bar{b}$ += $a\,\bar{p}$
$q = a^k$, k fijo	a^k	$\bar{a}$ += $k\,a^{k-1}\,\bar{q}$
$t = \tanh a$	$\tanh a$	$\bar{a}$ += $(1 - t^2)\,\bar{t}$
$e = \exp a$	e^a	$\bar{a}$ += $e\,\bar{e}$
$l = \ln a$	$\ln a$	$\bar{a}$ += $\bar{l}/a$

El método `backward()` del nodo de salida construye un orden topológico con la proposición 3.8, pone su propio `grad` a 1 y llama a `_backward` de cada nodo en orden inverso. Por el teorema 3.2, al terminar cada `grad` contiene la derivada de la salida respecto a ese nodo. El núcleo del sistema se muestra en la figura 6.1.

Las demás operaciones de la tabla se escriben como `__mul__`. Si una misma variable aparece en los dos factores, como en `x * x`, las dos líneas de `_backward` suman cada una su contribución a `x.grad`, y el resultado es $2x$, como exige la acumulación del tema 3.

Proposición 6.2 (Operaciones derivadas). *Si la resta, la negación y la división se definen como* $-a = (-1) \cdot a$, $a - b = a + (-1) \cdot b$ *y* $a/b = a \cdot b^{-1}$, *el sistema calcula correctamente sus derivadas sin reglas propias. Por ejemplo, para* $f = a/b$ *se obtiene* $\bar{a} = 1/b$ *y* $\bar{b} = -a/b^2$.

```
class Valor:
    def __init__(self, dato, _prev=()):
        self.dato, self.grad = dato, 0.0
        self._prev, self._backward = _prev, lambda: None

    def __mul__(self, otro):
        sal = Valor(self.dato * otro.dato, (self, otro))
        def _backward():
            self.grad += otro.dato * sal.grad
            otro.grad += self.dato * sal.grad
        sal._backward = _backward
        return sal

    def backward(self):
        orden, vistos = [], set()
        def visitar(v):
            if v not in vistos:
                vistos.add(v)
                for u in v._prev:
                    visitar(u)
                orden.append(v)
        visitar(self)
        self.grad = 1.0
        for v in reversed(orden):
            v._backward()
```

Figura 6.1: Núcleo de un autograd escalar: el nodo, la regla local del producto y el método `backward`, que construye un orden topológico (proposición 3.8) y lo recorre al revés.

Demostración. Cada operación derivada es una composición de operaciones de la tabla, y el teorema 3.2 se aplica a cualquier traza. Para $f = a \cdot q$ con $q = b^{-1}$: el nodo producto da $\bar{a} = q = 1/b$ y $\bar{q} = a$; el nodo potencia, con $k = -1$, da $\bar{b} = -b^{-2}\,\bar{q} = -a/b^2$. ■

Ejemplo 6.3 (Entropía cruzada con piezas elementales). Con logits $z = (2, 1, 0)$ y clase correcta 0, se construye $L = -\ln\big(e^{z_0}/(e^{z_0} + e^{z_1} + e^{z_2})\big)$ con nodos `exp`, suma, potencia -1, producto y `ln`. La llamada `L.backward()` da $L \approx 0{,}4076$ y $\nabla_z L \approx (-0{,}3348;\ 0{,}2447;\ 0{,}0900)$, que es $\mathrm{softmax}(z) - e_0$, el resultado del corolario 4.9 obtenido sin deducirlo.

Con estos nodos se construye una red: una neurona calcula $\tanh\big(\sum_i w_i x_i + b\big)$; una capa es una lista de neuronas, y un perceptrón multicapa, una lista de capas. Un perceptrón $3 \to 4 \to 4 \to 1$ tiene $(3\cdot 4+4)+(4\cdot 4+4)+(4\cdot 1+1) = 41$ parámetros. Cada paso de descenso de gradiente tiene tres fases: calcular la pérdida; poner a cero todos los `grad` y llamar a `backward`; y actualizar cada parámetro con $p \leftarrow p - \eta\,\bar{p}$. Si se omite la puesta a cero, los adjuntos de pasos sucesivos se suman por la acumulación de la definición 3.1.

Observación 6.4 (Límite del enfoque escalar). Un producto de dos matrices 1000×1000 ejecuta 10^9 multiplicaciones y casi otras tantas sumas: un autograd escalar crearía del orden de $2 \cdot 10^9$ nodos, cada uno con su objeto de Python. Los sistemas reales usan *tensores*: un nodo por operación sobre arrays enteros, con un VJP vectorizado como los de la proposición 4.6 y la proposición 4.8, que se ejecuta en una GPU. La tarea complementaria del curso 11-785 [4, §1 y §3.2] subraya este cambio de granularidad: el autograd trabaja a nivel de operación, no de capa, y cada operación necesita su función hacia atrás.

6.2 PyTorch

PyTorch [19] es, en esencia, el sistema anterior con tensores, ejecución en GPU y cientos de operaciones con su VJP. Es la herramienta del curso a partir de este módulo. El código de referencia publicado para los modelos LLaMA de Meta usa PyTorch [16]; el informe técnico de Gemini indica que se entrenó con JAX y Pathways [8], y el código de los pesos abiertos de Grok-1, un modelo de 314 000 millones de parámetros, también es de JAX [11]. Anthropic no ha publicado los detalles de su pila de software para Claude.

Pieza de PyTorch	Función	Equivalente escalar
`Tensor` con `requires_grad`	array que registra sus operaciones	`Valor`
`grad_fn`, `backward()`	nodo del grafo y modo inverso	`_backward`, `backward()`
`nn.Module`	agrupa parámetros, define `forward`	neurona, capa, MLP
`torch.optim` (SGD, AdamW)	aplica la actualización	$p \leftarrow p - \eta\,\bar{p}$

La nota «Autograd mechanics» de la documentación [21] precisa varios comportamientos que se siguen de la teoría de los temas anteriores.

(a) *Grafo dinámico.* El grafo se reconstruye desde cero en cada iteración, al ejecutar el código; por eso se puede usar el control de flujo de Python, y las derivadas son las de la traza ejecutada (tema 1).

(b) *Acumulación.* Durante `backward()`, solo los tensores *hoja* (los creados por el usuario) con `requires_grad=True` reciben gradiente, y este se *suma* a su atributo `.grad`. Por eso cada paso empieza con `opt.zero_grad()`.

(c) *Tensores guardados.* Cada operación guarda los tensores que necesita su VJP. Tras la pasada hacia atrás, esos tensores se liberan; una segunda llamada a `backward()` sobre el mismo grafo produce un error, salvo que la primera se haya hecho con `retain_graph=True`.

(d) *Desactivar el registro.* Dentro de `with torch.no_grad():` las operaciones no crean nodos ni guardan tensores; se usa para evaluar en validación y para actualizar los parámetros sin registrar la actualización.

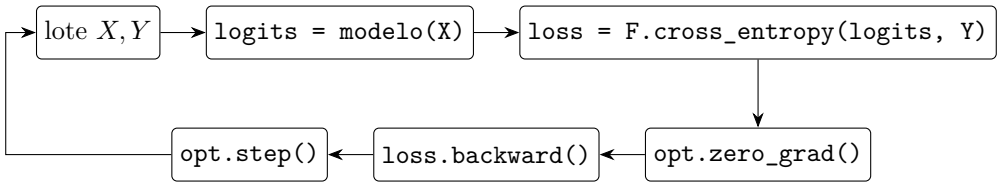


Figura 6.2: Un paso de entrenamiento en PyTorch: pasada hacia delante, pérdida, puesta a cero de los gradientes, modo inverso y actualización.

Ejemplo 6.5 (El modelo de caracteres del módulo 9). El modelo de lenguaje neuronal de caracteres del módulo 9, con vocabulario $V = 28$, contexto de $B = 3$ caracteres, embeddings de dimensión $d = 10$ y una capa oculta de $H = 64$ unidades ReLU, se escribe así:

```
class MLPCaracteres(nn.Module):
    def __init__(self, V, d=10, B=3, H=64):
        super().__init__()
        self.emb = nn.Embedding(V, d)       # matriz C
        self.oculta = nn.Linear(B * d, H)   # W1, b1
        self.salida = nn.Linear(H, V)       # W2, b2
    def forward(self, X):
        E = self.emb(X).view(X.shape[0], -1)
        return self.salida(F.relu(self.oculta(E)))
```

Tiene $28\cdot 10+(30\cdot 64+64)+(64\cdot 28+28) = 280+1984+1820 = 4084$ parámetros, como en el módulo 9, y la función de retropropagación escrita allí a mano desaparece. En la práctica del módulo, con descenso de gradiente de lote completo ($\eta = 0{,}5$, decaimiento $\lambda = 0{,}005$) durante 1000 pasos, la perplejidad de validación baja a 10,31, frente a 12,42 del modelo de bigramas; difiere del 10,45 del módulo 9 porque `nn.Linear` se inicializa de otra forma.

Observación 6.6 (Dos detalles de convención). `nn.Linear(n, m)` guarda su peso con forma (m, n) y calcula $XW^\top + b$, de modo que `weight` es la traspuesta de la W del módulo 9. Y `F.cross_entropy` recibe logits, no probabilidades: aplica internamente una log-softmax estable (módulo 11) y su VJP es el del corolario 4.9.

6.3 Acumulación de gradientes

La acumulación en `.grad`, que obliga a llamar a `zero_grad()`, es también una herramienta.

Proposición 6.7 (Microlotes). *Sea $\mathcal{L}(\theta) = \frac{1}{|S|}\sum_{s\in S}\ell_s(\theta)$ la pérdida media sobre un lote S, partido en k microlotes disjuntos $S_1,\dots,S_k$ del mismo tamaño. Si, sin poner a cero los gradientes entre ellos, se llama a `backward()` sobre $\mathcal{L}_r/k$ para cada r, con $\mathcal{L}_r = \frac{1}{|S_r|}\sum_{s\in S_r}\ell_s$, el gradiente acumulado es $\nabla\mathcal{L}(\theta)$.*

Demostración. Tras las k llamadas, el acumulador vale $\sum_r \nabla(\mathcal{L}_r/k) = \frac{1}{k}\sum_r \nabla\mathcal{L}_r$ por linealidad de la derivada. Como $|S_r| = |S|/k$, $\frac{1}{k}\mathcal{L}_r = \frac{1}{|S|}\sum_{s\in S_r}\ell_s$, y sumando sobre r se obtiene $\mathcal{L}$. ∎

Así se entrenan los LLM con lotes de millones de tokens que no caben a la vez en la memoria de una GPU: se procesan microlotes sucesivos, se acumulan sus gradientes y se da un solo paso del optimizador. Si los microlotes tuvieran tamaños distintos, habría que ponderar cada $\mathcal{L}_r$ por $|S_r|/|S|$ en lugar de por $1/k$.

Lecturas recomendadas para este tema

- Karpathy [14], micrograd, ficheros `engine.py` y `nn.py`.
- Paszke *et al.* [19], «PyTorch: An Imperative Style, High-Performance Deep Learning Library».
- Documentación de PyTorch [21], nota «Autograd mechanics», apartados «How autograd encodes the history» y «Locally disabling gradient computation».
- Zhang *et al.* [23], §2.5 «Automatic Differentiation», §2.5.1–2.5.4.
- CMU 11-785 [4], tarea complementaria «An Introduction to Autograd», §2 y §3.

Ejercicios del tema 6

Ejercicio 6.1 ⋆ En PyTorch, ¿qué ocurre si se olvida `opt.zero_grad()` en el bucle de entrenamiento? ¿Para qué sirve `torch.no_grad()` al evaluar en validación?

Ejercicio 6.2 ⋆ Con $x = 3$ (`requires_grad=True`) e $y = x \cdot x$, se ejecuta `y.backward()` dos veces seguidas. (a) ¿Qué ocurre? (b) ¿Y si en la primera llamada se pasa el argumento `retain_graph=True`? ¿Cuánto vale `x.grad` al final y por qué?

Fuente: adaptado de Zhang *et al.* [23], §2.5 «Automatic Differentiation», ejercicio 2.

Ejercicio 6.3 ⋆⋆ Escriba la función hacia atrás de cada una de las operaciones siguientes, que recibe el adjunto g (o G) de la salida y devuelve los de las entradas: $a - b$, $a \cdot b$, a/b, $\ln a$, $\exp a$ y el producto de matrices AB. Aplíquelas, en una cinta recorrida al revés, a $f = \ln(a/b) - \exp(a \cdot b)$ en $(a, b) = (2, 1)$ y compruebe el resultado derivando f.

Fuente: adaptado de CMU 11-785, otoño de 2021 [4], tarea complementaria «An Introduction to Autograd», §3.2.1 (funciones `sub_backward`, `mul_backward`, `divide_backward`, `log_backward`, `exp_backward` y `matmul_backward`).

Ejercicio 6.4 ⋆⋆ Cuente los parámetros de `MLPCaracteres` con $V = 28$, $d = 8$, $B = 3$ y $H = 64$, separando embeddings, capa oculta y capa de salida.

Ejercicio 6.5 ⋆⋆ Una neurona $o = \tanh(wx + b)$ con $x = 1$, $w = 0{,}5$ y $b = 0$ se entrena con la pérdida $L = \frac{1}{2}(o - y)^2$ e $y = 1$. Calcule L, $\partial L/\partial w$ y $\partial L/\partial b$, aplique un paso de descenso con $\eta = 0{,}5$ y compruebe que la pérdida disminuye.

Ejercicio 6.6 ⋆⋆⋆ (a) Demuestre la proposición 6.7 y explique qué ocurre si se llama a `backward()` sobre $\mathcal{L}_r$ en lugar de sobre $\mathcal{L}_r/k$. (b) Compruébela con el modelo $\ell_s(w) = \frac{1}{2}(wx_s - y_s)^2$, los datos $x = (1, 2, 3, 4)$, $y = (2, 3, 7, 8)$, $w = 1$ y dos microlotes de dos ejemplos.

Apéndice A

Soluciones de los ejercicios

Cada solución se presenta en cinco partes: datos, planteamiento, desarrollo, resultado y comprobación. En los pasos intermedios se redondea a cuatro cifras significativas salvo que se indique otra cosa; los cálculos con números duales y con enteros son exactos. Todos los resultados numéricos se han comprobado con Python y PyTorch en doble precisión.

A.1 Tema 1

Solución del ejercicio 1.1

Datos. la traza del ejemplo 1.5: v_{-1}, v_0 (entradas) y $v_1, \ldots, v_5$.

Planteamiento. contar un nodo por variable y una arista por cada par $j \prec i$ (definición 1.3); un orden topológico solo exige que cada nodo vaya detrás de sus argumentos (definición 3.7).

Desarrollo.

1. Nodos: $v_{-1}, v_0, v_1, \ldots, v_5$, en total 7.
2. Aristas: v_1 tiene un argumento, v_2 dos, v_3 uno, v_4 dos y v_5 dos: $1 + 2 + 1 + 2 + 2 = 8$.
3. Sucesores: v_{-1} alimenta a v_1 y v_2; v_0, a v_2 y v_3. Los demás nodos tienen un solo sucesor, salvo v_5, que no tiene ninguno.
4. Otro orden topológico: $v_0, v_{-1}, v_3, v_2, v_1, v_4, v_5$. Cada nodo aparece tras sus argumentos: v_3 tras v_0; v_2 tras v_{-1} y v_0; v_1 tras v_{-1}; v_4 tras v_1 y v_2; v_5 tras v_4 y v_3.

Resultado. **7 nodos y 8 aristas;** v_{-1} **y** v_0 **tienen dos sucesores**; un orden alternativo es $v_0, v_{-1}, v_3, v_2, v_1, v_4, v_5$.

Comprobación. la suma de los grados de entrada de los nodos interiores coincide con el número de aristas: 8.

Solución del ejercicio 1.2

Datos. diferencias centradas con $h = 10^{-5}$ y con $h = 10^{-15}$, AD; doble precisión, $\mathbf{u} \approx 1{,}1 \cdot 10^{-16}$.

Planteamiento. el error de la diferencia centrada es truncamiento, de orden h^2, más redondeo, de orden $\mathbf{u}/h$ (módulo 4, tema 6); la AD solo arrastra el redondeo de cada operación.

Desarrollo.

1. $h = 10^{-15}$: truncamiento del orden de 10^{-30}, despreciable; redondeo del orden de $10^{-16}/10^{-15} = 10^{-1}$. La resta $f(x+h) - f(x-h)$ pierde casi todas las cifras.
2. $h = 10^{-5}$: truncamiento del orden de $(10^{-5})^2 = 10^{-10}$ y redondeo del orden de $10^{-16}/10^{-5} = 10^{-11}$. Error total del orden de 10^{-10}.

3. AD: aplica reglas exactas, como $(e^x)' = e^x$, a los valores; el error relativo es del orden de $\mathbf{u}$.
4. Con $f = e^x$ en $x = 1$: el error medido es $5{,}9 \cdot 10^{-11}$ con $h = 10^{-5}$ y $0{,}17$ con $h = 10^{-15}$.

Resultado. **de menor a mayor precisión: $h = 10^{-15}$, $h = 10^{-5}$, AD.**

Comprobación. los errores medidos siguen el orden previsto: $0{,}17 \gg 5{,}9 \cdot 10^{-11} \gg 10^{-16}$.

Solución del ejercicio 1.3

Datos. $f(x) = \ln(x^2) \cdot \operatorname{sen} x + x^{-1}$, $x = 2$.

Planteamiento. descomponer en operaciones de la definición 1.1: un nodo por cada potencia, logaritmo, seno, producto y suma.

Desarrollo.

1. (a) Nodos: $a = x^2$, $b = \ln a$, $c = \operatorname{sen} x$, $d = b \cdot c$, $e = x^{-1}$, $f = d + e$. Aristas: $x \to a \to b \to d$, $x \to c \to d$, $x \to e$, $d \to f$, $e \to f$. La entrada x tiene tres sucesores: a, c y e.
2. (b) $a = 2^2 = 4$.
3. $b = \ln 4 = 1{,}386$.
4. $c = \operatorname{sen} 2 = 0{,}9093$.
5. $d = 1{,}386 \cdot 0{,}9093 = 1{,}261$.
6. $e = 1/2 = 0{,}5$.
7. $f = 1{,}261 + 0{,}5 = 1{,}761$.

Resultado. **grafo con seis nodos interiores y $f(2) \approx 1{,}761$.**

Comprobación. evaluando directamente, $\ln 4 \cdot \operatorname{sen} 2 + 0{,}5 = 1{,}2606 + 0{,}5 = 1{,}7606$.

Solución del ejercicio 1.4

Datos. $f(x) = e^x$; $h = 10^{-1}$ y $h = 10^{-8}$; $x = 1$ para las cifras.

Planteamiento. usar $e^{x+ih} = e^x(\cos h + i \operatorname{sen} h)$ y comparar con la diferencia centrada $\big(e^{x+h} - e^{x-h}\big)/(2h) = e^x \operatorname{senh}(h)/h$.

Desarrollo.

1. $\operatorname{Im} e^{x+ih} = e^x \operatorname{sen} h$, luego $\operatorname{Im} f(x+ih)/h = e^x \operatorname{sen}(h)/h$.
2. Error relativo respecto a $f'(x) = e^x$: $1 - \operatorname{sen}(h)/h$. Con $\operatorname{sen} h = h - h^3/6 + O(h^5)$, es $h^2/6 + O(h^4)$.
3. $h = 10^{-1}$: paso complejo $1 - \operatorname{sen}(0{,}1)/0{,}1 = 1{,}666 \cdot 10^{-3}$; centrada $\operatorname{sinh}(0{,}1)/0{,}1 - 1 = 1{,}668 \cdot 10^{-3}$. Domina el truncamiento y ambos son casi iguales.
4. $h = 10^{-8}$: paso complejo, $h^2/6 \approx 1{,}7 \cdot 10^{-17}$, por debajo de $\mathbf{u}$; el valor calculado coincide con e en todas las cifras. Centrada: truncamiento $1{,}7 \cdot 10^{-17}$, pero el redondeo, de orden $\mathbf{u}/h \approx 10^{-8}$, da un error medido de $2{,}4 \cdot 10^{-9}$.

Resultado. **error relativo** $1 - \operatorname{sen}(h)/h \approx h^2/6$: $1{,}67 \cdot 10^{-3}$ con $h = 0{,}1$ en ambos métodos; con $h = 10^{-8}$, error nulo en doble precisión con paso complejo y $2{,}4 \cdot 10^{-9}$ con la centrada.

Comprobación. el paso complejo no resta cantidades próximas, de modo que su error decrece con h sin límite (observación 1.8); la centrada alcanza su mínimo hacia $h \approx 10^{-5}$.

Solución del ejercicio 1.5

Datos. $l_1 = x = 0{,}2$, $l'_1 = 1$; $l_{k+1} = 4l_k(1 - l_k)$, $l'_{k+1} = 4l'_k(1 - 2l_k)$.

Planteamiento. propagar el par (valor, derivada) paso a paso, como en el modo directo.

Desarrollo.

1. $l_2 = 4 \cdot 0{,}2 \cdot 0{,}8 = 0{,}64$; $l'_2 = 4 \cdot 1 \cdot (1 - 0{,}4) = 2{,}4$.
2. $l_3 = 4 \cdot 0{,}64 \cdot 0{,}36 = 0{,}9216$; $l'_3 = 4 \cdot 2{,}4 \cdot (1 - 1{,}28) = 9{,}6 \cdot (-0{,}28) = -2{,}688$.
3. Comprobación simbólica: $l_3 = 16(x - x^2)(1 - 2x)^2$, y por la regla del producto $l'_3 = 16\big[(1 - 2x)^3 - 4(x - x^2)(1 - 2x)\big] = 16(1 - 2x)(1 - 8x + 8x^2)$.
4. En $x = 0{,}2$: $16 \cdot 0{,}6 \cdot (1 - 1{,}6 + 0{,}32) = 9{,}6 \cdot (-0{,}28) = -2{,}688$.

Resultado. $l_2 = 0{,}64$**,** $l'_2 = 2{,}4$**,** $l_3 = 0{,}9216$**,** $l'_3 = -2{,}688$**.**

Comprobación. $l_3 = 16 \cdot 0{,}2 \cdot 0{,}8 \cdot 0{,}36 = 0{,}9216$ coincide con el valor de la recurrencia.

Solución del ejercicio 1.6

Datos. $l_1 = x$, $l_{k+1} = 4l_k - 4l_k^2$.

Planteamiento. inducción sobre k para el grado; recuento directo de operaciones para el par (l_k, l'_k).

Desarrollo.

1. Base: $l_1 = x$ tiene grado $1 = 2^0$ y coeficiente principal $c_1 = 1 \neq 0$.
2. Hipótesis: l_k tiene grado 2^{k-1} y coeficiente principal $c_k \neq 0$.
3. Entonces l_k^2 tiene grado 2^k y coeficiente principal c_k^2, mientras que $4l_k$ tiene grado $2^{k-1} < 2^k$. El término de grado 2^k de l_{k+1} es $-4c_k^2 x^{2^k}$, con $c_{k+1} = -4c_k^2 \neq 0$: grado exactamente 2^k.
4. Un polinomio de grado 2^{k-1} tiene $2^{k-1}+1$ coeficientes, de modo que la forma desarrollada de l_k, y con ella la de l'_k, crece exponencialmente con k.
5. Par (l_{k+1}, l'_{k+1}) a partir de (l_k, l'_k): $t = 1 - l_k$ (1 operación); $l_{k+1} = 4 \cdot l_k \cdot t$ (2); $s = t - l_k = 1 - 2l_k$ (1); $l'_{k+1} = 4 \cdot l'_k \cdot s$ (2). Total: 6 operaciones por paso.
6. De $(l_1, l'_1) = (x, 1)$ a (l_k, l'_k) hay $k - 1$ pasos: a lo sumo $6(k - 1)$ operaciones.

Resultado. $\deg l_k = 2^{k-1}$**, mientras que el modo directo calcula** (l_k, l'_k) **con** $6(k - 1)$ **operaciones**: coste lineal frente a una expresión de tamaño exponencial.

Comprobación. $l_3 = 16x(1-x)(1-2x)^2$ tiene grado $1+1+2 = 4 = 2^2$, con coeficiente principal $16 \cdot (-1) \cdot 4 = -64 = -4c_2^2$, pues $c_2 = -4$.

A.2 Tema 2

Solución del ejercicio 2.1

Datos. $(3 + \varepsilon)^2$; $f\colon \mathbb{R}^{1000} \to \mathbb{R}$.

Planteamiento. por el teorema 2.9, la parte dual de $f(a+\varepsilon)$ es $f'(a)$; por el corolario 2.4, cada pasada da una derivada direccional.

Desarrollo.

1. $(3+\varepsilon)^2 = 9 + 6\varepsilon + \varepsilon^2 = 9 + 6\varepsilon$.
2. La parte dual, 6, es la derivada de x^2 en $x = 3$.
3. El gradiente de $f\colon \mathbb{R}^{1000} \to \mathbb{R}$ tiene 1000 componentes, y cada pasada en modo directo con semilla e_k da una: 1000 pasadas.
4. Una derivada direccional $\nabla f^\top v$ es exactamente lo que da una pasada con semilla v: una pasada.

Resultado. **(a) parte dual 6, la derivada de x^2 en 3; (b) 1000 pasadas para el gradiente y una para una derivada direccional.**
Comprobación. $\frac{d}{dx}x^2 = 2x = 6$ en $x = 3$.

Solución del ejercicio 2.2

Datos. $f(x) = x^4 - 3x$, $x = 1$.
Planteamiento. evaluar $f(1+\varepsilon)$ con $\varepsilon^2 = 0$ y leer la parte real y la dual.
Desarrollo.

1. $x^2 = (1+\varepsilon)(1+\varepsilon) = 1 + 2\varepsilon$.
2. $x^4 = (1+2\varepsilon)(1+2\varepsilon) = 1 + 4\varepsilon$.
3. $3x = 3 + 3\varepsilon$.
4. $f = (1+4\varepsilon) - (3+3\varepsilon) = -2 + \varepsilon$.

Resultado. $f(1) = -2$ **y** $f'(1) = 1$**.**
Comprobación. $f'(x) = 4x^3 - 3$ vale $4 - 3 = 1$ en $x = 1$.

Solución del ejercicio 2.3

Datos. $g(x) = x/(1+x^2)$, $x = 2$; $1/(a+b\varepsilon) = 1/a - (b/a^2)\varepsilon$.
Planteamiento. calcular el denominador como dual, invertirlo con la proposición 2.6 y multiplicar por x.
Desarrollo.

1. $x^2 = (2+\varepsilon)^2 = 4 + 4\varepsilon$, luego $1 + x^2 = 5 + 4\varepsilon$.
2. Inverso con $a = 5$, $b = 4$: $\frac{1}{5} - \frac{4}{25}\varepsilon = 0{,}2 - 0{,}16\varepsilon$.
3. $(2+\varepsilon)(0{,}2 - 0{,}16\varepsilon) = 0{,}4 + \big(2\cdot(-0{,}16) + 0{,}2\big)\varepsilon = 0{,}4 - 0{,}12\varepsilon$.

Resultado. $g(2) = 2/5 = 0{,}4$ **y** $g'(2) = -3/25 = -0{,}12$**.**
Comprobación. $g'(x) = (1-x^2)/(1+x^2)^2$ vale $(1-4)/25 = -3/25$ en $x = 2$.

Solución del ejercicio 2.4

Datos. $h(x) = \tanh(x^2)$, $x = 1$; $\tanh(a+b\varepsilon) = \tanh a + (1 - \tanh^2 a)\, b\, \varepsilon$.
Planteamiento. evaluar x^2 en duales y aplicar la extensión dual de tanh (definición 2.7).
Desarrollo.

1. $x^2 = (1+\varepsilon)^2 = 1 + 2\varepsilon$.
2. $\tanh 1 = 0{,}76159$ y $1 - \tanh^2 1 = 1 - 0{,}58002 = 0{,}41997$.

3. $\tanh(1+2\varepsilon) = 0{,}76159 + 0{,}41997 \cdot 2\,\varepsilon = 0{,}7616 + 0{,}8399\,\varepsilon$.

Resultado. $h(1) \approx 0{,}7616$ **y** $h'(1) \approx 0{,}8399$**.**

Comprobación. por la regla de la cadena, $h'(x) = (1 - \tanh^2 x^2) \cdot 2x$, que en $x = 1$ vale $0{,}41997 \cdot 2 = 0{,}8399$.

Solución del ejercicio 2.5

Datos. $x_1 = 2 + \varepsilon_1$, $x_2 = 5 + \varepsilon_2$, $\varepsilon_r\varepsilon_s = 0$; $\cos 5 = 0{,}2837$.

Planteamiento. cada variable lleva un valor y dos partes duales; la parte en ε_r es la derivada respecto a x_r (observación 2.11).

Desarrollo.

1. $v_1 = \ln(2 + \varepsilon_1) = 0{,}6931 + 0{,}5\,\varepsilon_1$.
2. $v_2 = (2 + \varepsilon_1)(5 + \varepsilon_2) = 10 + 5\varepsilon_1 + 2\varepsilon_2$, porque $\varepsilon_1\varepsilon_2 = 0$.
3. $v_3 = \operatorname{sen}(5 + \varepsilon_2) = -0{,}9589 + 0{,}2837\,\varepsilon_2$.
4. $v_4 = v_1 + v_2 = 10{,}6931 + 5{,}5\,\varepsilon_1 + 2\,\varepsilon_2$.
5. $v_5 = v_4 - v_3 = 11{,}6521 + 5{,}5\,\varepsilon_1 + (2 - 0{,}2837)\,\varepsilon_2 = 11{,}6521 + 5{,}5\,\varepsilon_1 + 1{,}7163\,\varepsilon_2$.
6. Derivada direccional en $(1,1)^\top$: $\nabla f^\top (1,1)^\top = 5{,}5 + 1{,}7163 = 7{,}2163$.

Resultado. $\nabla f(2,5) = (5{,}5;\ 1{,}7163)^\top$ **en una pasada; derivada direccional** 7,2163**.**

Comprobación. coincide con el ejemplo 2.3 y con la traza adjunta del ejemplo 3.4; además, la semilla única $x = (2 + \varepsilon, 5 + \varepsilon)$ daría directamente 7,2163.

Solución del ejercicio 2.6

Datos. polinomios reales; aritmética de la definición 2.5.

Planteamiento. inducción sobre el grado con la descomposición $p(x) = x\,q(x) + c$; después, la regla de la cadena para la composición.

Desarrollo.

1. Base (grado 0): $p = c$ constante. $p(a + b\varepsilon) = c = c + 0 \cdot b\,\varepsilon$ y $p'(a) = 0$.
2. Hipótesis: todo polinomio q de grado a lo sumo n cumple $q(a + b\varepsilon) = q(a) + q'(a)b\varepsilon$.
3. Todo p de grado $n + 1$ se escribe $p(x) = x\,q(x) + c$ con q de grado n y $c = p(0)$.
4. Por la hipótesis, $p(a + b\varepsilon) = (a + b\varepsilon)\big(q(a) + q'(a)b\varepsilon\big) + c$.
5. Producto de duales: $(a + b\varepsilon)(q(a) + q'(a)b\varepsilon) = a\,q(a) + \big(a\,q'(a) + q(a)\big)b\,\varepsilon$.
6. Por la regla del producto, $p'(x) = q(x) + x\,q'(x)$, luego $p(a + b\varepsilon) = p(a) + p'(a)\,b\,\varepsilon$.
7. Composición: si f cumple la propiedad, $f(a + b\varepsilon) = f(a) + f'(a)b\,\varepsilon$. Aplicando la de g con $a' = f(a)$ y $b' = f'(a)b$: $g(f(a + b\varepsilon)) = g(f(a)) + g'(f(a))f'(a)\,b\,\varepsilon$.
8. Por la regla de la cadena, $g'(f(a))f'(a) = (g \circ f)'(a)$.

Resultado. **la propiedad vale para todo polinomio y se conserva por composición**; es el teorema 2.9 para funciones de una variable.

Comprobación. $p(x) = x^3$ en $a = 2$, $b = 1$: el ejemplo 2.10 dio $(2 + \varepsilon)^3 = 8 + 12\varepsilon$ y $p'(2) = 12$.

A.3 Tema 3

Solución del ejercicio 3.1

Datos. actualización $\bar{v}_j \mathrel{+}= \bar{v}_i \, \partial\varphi_i/\partial v_j$; ejemplo $y = x \cdot x$ con $x = 3$.

Planteamiento. por el teorema 3.2, el adjunto de una variable es la suma de las contribuciones de todas las operaciones que la usan.

Desarrollo.

1. El producto $y = a \cdot b$ con $a = b = x$ ejecuta $\bar{a} \mathrel{+}= b\,\bar{y}$ y $\bar{b} \mathrel{+}= a\,\bar{y}$, y ambas líneas actúan sobre $\bar{x}$.
2. Con acumulación: $\bar{x} = 0 + 3 \cdot 1 + 3 \cdot 1 = 6$.
3. Con asignación: la primera línea pone $\bar{x} = 3$ y la segunda lo sobrescribe con $\bar{x} = 3$.

Resultado. **con acumulación se obtiene $2x = 6$, correcto; con asignación, 3, erróneo.**

Comprobación. $\frac{d}{dx}x^2 = 6$ en $x = 3$. El ejercicio 3.7 caracteriza cuándo la asignación basta.

Solución del ejercicio 3.2

Datos. $f(x, y) = (x + y) \cdot (x \cdot y)$ en $(2, 3)$.

Planteamiento. nodos $s = x + y$, $p = x \cdot y$, $f = s \cdot p$; x e y alimentan a dos nodos y sus adjuntos acumulan dos contribuciones.

Desarrollo.

1. Hacia delante: $s = 5$, $p = 6$, $f = 30$.
2. $\bar{f} = 1$. Nodo $f = s \cdot p$: $\bar{s} = p = 6$, $\bar{p} = s = 5$.
3. Nodo $s = x + y$: $\bar{x} \mathrel{+}= 6$, $\bar{y} \mathrel{+}= 6$.
4. Nodo $p = x \cdot y$: $\bar{x} \mathrel{+}= \bar{p}\,y = 15$, luego $\bar{x} = 21$; $\bar{y} \mathrel{+}= \bar{p}\,x = 10$, luego $\bar{y} = 16$.

Resultado. $f = 30$**,** $\partial f/\partial x = 21$**,** $\partial f/\partial y = 16$**.**

Comprobación. $f = x^2y + xy^2$: $\partial_x f = 2xy + y^2 = 12 + 9 = 21$ y $\partial_y f = x^2 + 2xy = 4 + 12 = 16$.

Solución del ejercicio 3.3

Datos. $o = \tanh(n)$, $n = w_1x_1 + w_2x_2 + b$, $x = (2, 0)$, $w = (-1, 3)$, $b = 1$.

Planteamiento. como en el ejemplo 3.6: $\bar{n} = (1 - o^2)\bar{o}$ con $\bar{o} = 1$, y después las reglas de la suma y del producto.

Desarrollo.

1. $n = (-1) \cdot 2 + 3 \cdot 0 + 1 = -1$.
2. $o = \tanh(-1) = -0{,}76159$; $1 - o^2 = 0{,}41997 = \bar{n}$.
3. Suma: $\bar{b} = \bar{n} = 0{,}4200$.
4. Productos: $\bar{w}_1 = \bar{n}\,x_1 = 0{,}8399$; $\bar{w}_2 = \bar{n}\,x_2 = 0$; $\bar{x}_1 = \bar{n}\,w_1 = -0{,}4200$; $\bar{x}_2 = \bar{n}\,w_2 = 1{,}260$.

Resultado. $o \approx -0{,}7616$**;** $\partial o/\partial w_1 \approx 0{,}8399$**,** $\partial o/\partial w_2 = 0$**,** $\partial o/\partial b \approx 0{,}4200$**,** $\partial o/\partial x_1 \approx -0{,}4200$**,** $\partial o/\partial x_2 \approx 1{,}260$**.**

Comprobación. un peso cuya entrada es nula no recibe gradiente, y $\partial o/\partial w_1 = x_1\,\partial o/\partial b = 2 \cdot 0{,}41997$.

Solución del ejercicio 3.4

Datos. $f = a \cdot b + a$, $a = 2$, $b = 5$; nodos a, b, $m = a \cdot b$, $f = m + a$.

Planteamiento. m debe ir tras a y b; f, tras m y a.

Desarrollo.

1. Órdenes topológicos: solo pueden variar las posiciones relativas de a y b, que van antes que m, y f va al final: (a, b, m, f) y (b, a, m, f).
2. Hacia delante: $m = 10$, $f = 12$.
3. $\bar{f} = 1$. Nodo $f = m + a$: $\bar{m} = 1$, $\bar{a}$ += 1.
4. Nodo $m = a \cdot b$: $\bar{a}$ += $\bar{m}\,b = 5$, luego $\bar{a} = 6$; $\bar{b}$ += $\bar{m}\,a = 2$.

Resultado. **dos órdenes topológicos;** $f = 12$**,** $\partial f/\partial a = 6$**,** $\partial f/\partial b = 2$**.**

Comprobación. $\partial f/\partial a = b + 1 = 6$ y $\partial f/\partial b = a = 2$.

Solución del ejercicio 3.5

Datos. el grafo del ejercicio 1.3: $a = x^2$, $b = \ln a$, $c = \operatorname{sen} x$, $d = b \cdot c$, $e = 1/x$, $f = d + e$, en $x = 2$, con $a = 4$, $b = 1{,}386$, $c = 0{,}9093$.

Planteamiento. modo directo con $\dot{x} = 1$; modo inverso con $\bar{f} = 1$, sumando en $\bar{x}$ las tres contribuciones de a, c y e.

Desarrollo.

1. (a) Derivadas locales: $\partial a/\partial x = 2x = 4$; $\partial b/\partial a = 1/a = 0{,}25$; $\partial c/\partial x = \cos 2 = -0{,}4161$; $\partial d/\partial b = c = 0{,}9093$; $\partial d/\partial c = b = 1{,}386$; $\partial e/\partial x = -1/x^2 = -0{,}25$; $\partial f/\partial d = \partial f/\partial e = 1$.
2. (b) Directo: $\dot{a} = 4$; $\dot{b} = \dot{a}/a = 1$; $\dot{c} = -0{,}4161$; $\dot{d} = \dot{b}\,c + b\,\dot{c} = 0{,}9093 - 0{,}5769 = 0{,}3324$; $\dot{e} = -0{,}25$; $\dot{f} = 0{,}3324 - 0{,}25 = 0{,}0824$.
3. (c) Inverso: $\bar{d} = \bar{e} = 1$; $\bar{b} = \bar{d}\,c = 0{,}9093$; $\bar{c} = \bar{d}\,b = 1{,}386$; $\bar{a} = \bar{b}/a = 0{,}2273$.
4. Contribuciones a $\bar{x}$: desde a, $0{,}2273 \cdot 4 = 0{,}9093$; desde c, $1{,}386 \cdot (-0{,}4161) = -0{,}5769$; desde e, $1 \cdot (-0{,}25) = -0{,}25$. Suma: $\bar{x} = 0{,}0824$.

Resultado. $f'(2) \approx 0{,}0824$ **por los dos modos.**

Comprobación. $f'(x) = \frac{2}{x}\operatorname{sen} x + \ln(x^2)\cos x - \frac{1}{x^2}$; en $x = 2$: $0{,}9093 - 0{,}5769 - 0{,}25 = 0{,}0824$. PyTorch da 0,08240.

Solución del ejercicio 3.6

Datos. $x = (1, 2)$, $W_1 = \left(\begin{smallmatrix} 1 & 0 \\ -1 & 0{,}5 \end{smallmatrix}\right)$, $b_1 = (0{,}5;\ 0)$, $W_2 = \left(\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right)$, $y = 3$, $\lambda = 0{,}1$.

Planteamiento. el sesgo es un sumando más de z, luego $\bar{b}_1 = \bar{z}$; no aparece en s y no recibe término de regularización. Cada peso recibe la parte de L más λW, por la arista que lo une a s.

Desarrollo.

1. (a) Ecuaciones: $\bar{o} = o - y$; $\bar{W}_2 = h^\top \bar{o} + \lambda W_2$; $\bar{h} = \bar{o}\, W_2^\top$; $\bar{z} = \bar{h} \odot \mathbb{1}[z > 0]$; $\bar{W}_1 = x^\top \bar{z} + \lambda W_1$; $\bar{b}_1 = \bar{z}$.
2. (b) $xW_1 = (1 - 2,\ 0 + 1) = (-1, 1)$; $z = (-0{,}5;\ 1)$; $h = (0, 1)$.
3. $o = hW_2 = 1$; $L = \frac{1}{2}(1 - 3)^2 = 2$.
4. $\|W_1\|_F^2 = 1 + 0 + 1 + 0{,}25 = 2{,}25$; $\|W_2\|_F^2 = 5$; $s = 0{,}05 \cdot 7{,}25 = 0{,}3625$; $J = 2{,}3625$.
5. $\bar{o} = -2$; $\bar{W}_2 = (0;\ -2)^\top + (0{,}2;\ 0{,}1)^\top = (0{,}2;\ -1{,}9)^\top$.
6. $\bar{h} = -2 \cdot (2, 1) = (-4, -2)$; como $z_1 < 0$, $\bar{z} = (0, -2)$.
7. $x^\top \bar{z} = \left(\begin{smallmatrix} 0 & -2 \\ 0 & -4 \end{smallmatrix}\right)$ y $\bar{W}_1 = \left(\begin{smallmatrix} 0 & -2 \\ 0 & -4 \end{smallmatrix}\right) + \left(\begin{smallmatrix} 0{,}1 & 0 \\ -0{,}1 & 0{,}05 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 0{,}1 & -2 \\ -0{,}1 & -3{,}95 \end{smallmatrix}\right)$.
8. $\bar{b}_1 = \bar{z} = (0, -2)$.

Resultado. $J = 2{,}3625$**;** $\nabla_{W_2} J = (0{,}2;\ -1{,}9)^\top$**;** $\nabla_{W_1} J = \left(\begin{smallmatrix} 0{,}1 & -2 \\ -0{,}1 & -3{,}95 \end{smallmatrix}\right)$**;** $\nabla_{b_1} J = (0, -2)$**.**

Comprobación. la primera columna de $\nabla_{W_1} J$ es solo λW_1, porque la unidad 1 está inactiva. PyTorch da los mismos valores.

Solución del ejercicio 3.7

Datos. el algoritmo de la figura 3.1 con asignación en lugar de acumulación.

Planteamiento. si cada variable es argumento de una sola operación, cada acumulador recibe una única contribución, y sumarla a 0 equivale a asignarla.

Desarrollo.

1. Sea $j < \ell$. Si v_j es argumento de una única operación i, y figura una sola vez en su lista de argumentos, la versión con acumulación ejecuta sobre $\bar{v}_j$ una sola actualización, $\bar{v}_j \leftarrow 0 + \bar{v}_i\, \partial\varphi_i / \partial v_j$, que coincide con la asignación.
2. La salida v_ℓ no es argumento de ninguna operación y conserva la semilla $\bar{v}_\ell = 1$ en ambas versiones.
3. Por tanto, las dos versiones ejecutan las mismas operaciones con los mismos valores, y por el teorema 3.2 la versión con asignación es correcta.
4. Contraejemplo con un nodo de dos sucesores: $u = 2x$, $z = u^2 + u$ con $x = 3$, es decir, $y = u^2$ y $z = y + u$. Con asignación, el nodo z pone $\bar{y} = 1$ y $\bar{u} = 1$; el nodo y *sustituye* $\bar{u}$ por $2u \cdot \bar{y} = 12$, y el nodo u da $\bar{x} = 2 \cdot 12 = 24$.
5. El valor correcto es $dz/dx = 8x + 2 = 26$: la asignación ha perdido la contribución $\bar{u} = 1$ del camino directo.

Resultado. **la asignación es correcta en un árbol; con un nodo usado dos veces puede fallar**, como en el ejemplo, que da 24 en lugar de 26.

Comprobación. con acumulación, $\bar{u} = 1 + 12 = 13$ y $\bar{x} = 26$ (ejemplo 3.9).

A.4 Tema 4

Solución del ejercicio 4.1

Datos. $f\colon \mathbb{R}^{1000} \to \mathbb{R}$ $(n = 1000,\ m = 1)$; $g\colon \mathbb{R} \to \mathbb{R}^{1000}$ $(n = 1,\ m = 1000)$.

Planteamiento. el modo directo da una columna del jacobiano por pasada y el inverso una fila (corolario 4.2).

Desarrollo.

1. J_f es 1×1000, una fila: el modo inverso necesita 1 pasada; el directo, 1000.
2. J_g es 1000×1, una columna: el modo directo necesita 1 pasada; el inverso, 1000.

Resultado. **f: modo inverso, 1 pasada; g: modo directo, 1 pasada.** El modo contrario costaría 1000 pasadas en ambos casos.

Comprobación. el número de pasadas es $\min(n, m) = 1$ en los dos casos, eligiendo el modo adecuado.

Solución del ejercicio 4.2

Datos. los dos modos de la AD aplicados a $f\colon \mathbb{R}^n \to \mathbb{R}^m$.

Planteamiento. comparar los modos según los tres criterios del enunciado, usando las corolario 2.4 y proposiciones 5.1 y 3.3 y la proposición 4.3.

Desarrollo.

1. *Datos intermedios.* El modo directo descarta cada tangente en cuanto deja de usarse y necesita memoria comparable a la de evaluar f. El inverso debe guardar los valores de toda la pasada hacia delante (proposición 5.1).
2. *Paralelismo.* En el modo directo, las n pasadas con semillas distintas son independientes y pueden ejecutarse en paralelo, o en una sola pasada vectorial (observación 2.11). En el inverso, la pasada hacia atrás no puede empezar hasta terminar la pasada hacia delante.
3. *Tamaños.* El directo propaga matrices $m' \times n$ o vectores de tamaño n por dirección; el inverso propaga vectores fila del tamaño de cada variable intermedia. Con una salida escalar y muchas entradas, el inverso multiplica siempre vectores por matrices (proposición 4.3).

Resultado. **directo cuando hay pocas entradas ($n \ll m$) o la memoria es crítica; inverso cuando hay pocas salidas ($m \ll n$), como en el entrenamiento, con una pérdida escalar.**

Comprobación. un LLM tiene $n \approx 10^{11}$ y $m = 1$: el inverso cuesta una pasada, el directo 10^{11}.

Solución del ejercicio 4.3

Datos. $c \in \mathbb{R}^{1\times 100}$; $A, B \in \mathbb{R}^{100\times 100}$.

Planteamiento. un producto $(p \times q)(q \times r)$ cuesta pqr multiplicaciones (proposición 4.3 con $L = 3$).

Desarrollo.

1. $c(BA)$: BA cuesta $100^3 = 1\,000\,000$; $c\,(BA)$ cuesta $100^2 = 10\,000$. Total: $1\,010\,000$.
2. $(cB)A$: cB cuesta $10\,000$ y da un vector 1×100; $(cB)A$ cuesta otros $10\,000$. Total: $20\,000$.
3. Cociente: $1\,010\,000/20\,000 = 50{,}5$.
4. En general: $(m^3 + m^2)/(2m^2) = (m + 1)/2$.

Resultado. 1 010 000 **frente a** 20 000 **multiplicaciones; cociente** $50{,}5 = (m+1)/2$ **con** $m = 100$**.**

Comprobación. con $m = 1000$ el cociente es $1001/2 = 500{,}5$: crece linealmente con m.

Solución del ejercicio 4.4

Datos. $X = (2, -1)$, $W = \left(\begin{smallmatrix} 1 & 2 & 0 \\ 0 & 1 & -1 \end{smallmatrix}\right)$, $\bar{Y} = (1, 0, 2)$.

Planteamiento. proposición 4.6: $\bar{X} = \bar{Y} W^\top$ y $\bar{W} = X^\top \bar{Y}$.

Desarrollo.

1. $Y = XW = (2 \cdot 1 + (-1) \cdot 0,\ 2 \cdot 2 + (-1) \cdot 1,\ 2 \cdot 0 + (-1) \cdot (-1)) = (2, 3, 1)$.
2. $W^\top = \left(\begin{smallmatrix} 1 & 0 \\ 2 & 1 \\ 0 & -1 \end{smallmatrix}\right)$; $\bar{X} = \bar{Y} W^\top = (1 \cdot 1 + 0 \cdot 2 + 2 \cdot 0,\ 1 \cdot 0 + 0 \cdot 1 + 2 \cdot (-1)) = (1, -2)$.
3. $\bar{W} = X^\top \bar{Y} = \left(\begin{smallmatrix} 2 \\ -1 \end{smallmatrix}\right)(1, 0, 2) = \left(\begin{smallmatrix} 2 & 0 & 4 \\ -1 & 0 & -2 \end{smallmatrix}\right)$.
4. Formas: $\bar{X}$ es 1×2, como X; $\bar{W}$ es 2×3, como W.
5. Comprobación directa: $L = \bar{Y} Y^\top = Y_1 + 2Y_3$ y $Y_3 = X_1 W_{13} + X_2 W_{23}$, luego $\partial L / \partial W_{13} = 2X_1 = 4$.

Resultado. $Y = (2, 3, 1)$**;** $\bar{X} = (1, -2)$**;** $\bar{W} = \left(\begin{smallmatrix} 2 & 0 & 4 \\ -1 & 0 & -2 \end{smallmatrix}\right)$**.**

Comprobación. la columna central de $\bar{W}$ es nula porque $\bar{Y}_2 = 0$, y la entrada $(1, 3)$ coincide con la derivada directa.

Solución del ejercicio 4.5

Datos. $N = 10^9$; 512×1024 tokens por paso; 10^5 pasos; pico $495 \cdot 10^{12}$ FLOP/s; utilización 0,5.

Planteamiento. FLOPs por paso $= (2N + 4N) \cdot$ tokens por paso (corolario 4.11); tiempo = FLOPs totales/FLOP/s efectivos.

Desarrollo.

1. Tokens por paso: $512 \cdot 1024 = 524\,288$.
2. FLOPs por paso: $6 \cdot 10^9 \cdot 524\,288 = 3{,}146 \cdot 10^{15}$.
3. Total: $3{,}146 \cdot 10^{15} \cdot 10^5 = 3{,}146 \cdot 10^{20}$ FLOPs.
4. Rendimiento efectivo: $0{,}5 \cdot 495 \cdot 10^{12} = 2{,}475 \cdot 10^{14}$ FLOP/s.
5. Tiempo: $3{,}146 \cdot 10^{20} / 2{,}475 \cdot 10^{14} = 1{,}271 \cdot 10^6$ s, es decir, $1{,}271 \cdot 10^6 / 3600 = 353{,}1$ h y $353{,}1/24 = 14{,}71$ días.

Resultado. $\approx 3{,}15 \cdot 10^{20}$ **FLOPs;** ≈ 353 **h, unos** 14,7 **días en una GPU.**

Comprobación. $D = 524\,288 \cdot 10^5 \approx 5{,}24 \cdot 10^{10}$ tokens y $6ND = 6 \cdot 10^9 \cdot 5{,}24 \cdot 10^{10} \approx 3{,}15 \cdot 10^{20}$.

Solución del ejercicio 4.6

Datos. $p_i = e^{z_i}/S$, $S = \sum_k e^{z_k}$; $z = (2, 1, 0)$, $y = 0$.

Planteamiento. derivar $\ln p_i$ en lugar de p_i, usando $\partial p_i / \partial z_j = p_i \, \partial \ln p_i / \partial z_j$.

Desarrollo.

1. (a) $\ln p_i = z_i - \ln S$ da $\partial \ln p_i / \partial z_j = \mathbb{1}[i = j] - e^{z_j}/S = \mathbb{1}[i = j] - p_j$.

2. Luego $\partial p_i/\partial z_j = p_i(\mathbb{1}[i=j] - p_j)$ y $\bar{z}_j = \sum_i \bar{p}_i p_i(\mathbb{1}[i=j] - p_j) = p_j\bar{p}_j - p_j\langle\bar{p}, p\rangle$, que es la proposición 4.8.
3. (b) Para $L = -\ln p_y$, $\bar{p} = -e_y/p_y$ y $\langle\bar{p}, p\rangle = -1$. Entonces $\bar{z}_j = p_j(\bar{p}_j + 1)$: vale $p_y - 1$ si $j = y$ y p_j si $j \neq y$, es decir, $\bar{z} = p - e_y$.
4. Suma de las componentes: $\sum_j (p_j - \mathbb{1}[j=y]) = 1 - 1 = 0$.
5. (c) $e^z = (7{,}389;\ 2{,}718;\ 1)$, con $S = 11{,}107$; $p = (0{,}6652;\ 0{,}2447;\ 0{,}0900)$.
6. $\nabla_z L = (0{,}6652 - 1;\ 0{,}2447;\ 0{,}0900) = (-0{,}3348;\ 0{,}2447;\ 0{,}0900)$.
7. $L = -\ln 0{,}6652 = 0{,}4076$.

Resultado. $\nabla_z L = \mathrm{softmax}(z) - e_y$**; con** $z = (2, 1, 0)$ **e** $y = 0$**,** $\nabla_z L \approx (-0{,}3348;\ 0{,}2447;\ 0{,}0900)$ **y** $L \approx 0{,}4076$**.**

Comprobación. $-0{,}3348 + 0{,}2447 + 0{,}0900 = -0{,}0001 \approx 0$, por redondeo; también $L = -2 + \ln 11{,}107 = 0{,}4076$. Coincide con el ejemplo 6.3 y con `F.cross_entropy`.

A.5 Tema 5

Solución del ejercicio 5.1

Datos. $f\colon \mathbb{R}^n \to \mathbb{R}$; gradiente con un coste $O(\mathrm{ops}(f))$ (teorema 4.1).

Planteamiento. el hessiano tiene n^2 entradas, frente a las n del gradiente, y cada columna se obtiene con un producto hessiana-vector (proposición 5.4).

Desarrollo.

1. La segunda derivada se obtiene aplicando la AD al programa que calcula el gradiente, que ya es una o dos veces más largo que el de f y contiene las derivadas locales.
2. Un producto $H_f v$ cuesta, por la proposición 5.4, una constante por $\mathrm{ops}(f)$: del mismo orden que el gradiente.
3. El hessiano completo necesita n productos hessiana-vector, uno por columna: coste del orden de $n \cdot \mathrm{ops}(f)$, y n^2 números que guardar.
4. Además, el programa del gradiente debe conservar su propio grafo para poder derivarlo de nuevo, lo que aumenta la memoria (en PyTorch, `create_graph=True`).

Resultado. **un producto hessiana-vector cuesta lo mismo que un gradiente, salvo una constante; el hessiano completo cuesta** n **veces más y ocupa** n^2 **números.**

Comprobación. con $n = 10^9$, el hessiano tendría 10^{18} entradas, inabordable; los métodos que usan curvatura emplean productos hessiana-vector.

Solución del ejercicio 5.2

Datos. $f = x_1^2 x_2 + x_2^3$, punto $(1, 2)$, $v = (0, 1)^\top$.

Planteamiento. calcular $\nabla f = (2x_1x_2,\ x_1^2 + 3x_2^2)$ con $x_1 = 1$ y $x_2 = 2 + \varepsilon$; la parte dual es Hv (proposición 5.4).

Desarrollo.

1. Primera componente: $2 \cdot 1 \cdot (2 + \varepsilon) = 4 + 2\varepsilon$.

2. $(2+\varepsilon)^2 = 4 + 4\varepsilon$, luego $3x_2^2 = 12 + 12\varepsilon$.
3. Segunda componente: $1 + 12 + 12\varepsilon = 13 + 12\varepsilon$.
4. Lectura: $\nabla f(1,2) = (4, 13)$ y $Hv = (2, 12)$.
5. Hessiano: $H = \left(\begin{smallmatrix} 2x_2 & 2x_1 \\ 2x_1 & 6x_2 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 4 & 2 \\ 2 & 12 \end{smallmatrix}\right)$, cuya segunda columna es $(2, 12)$.

Resultado. $\nabla f(1,2) = (4, 13)$ **y** $H_f(1,2)\, v = (2, 12)$**.**

Comprobación. PyTorch, derivando el gradiente con `create_graph=True`, da los mismos vectores.

Solución del ejercicio 5.3

Datos. $x_{t+1} = ax_t + b$, $t = 0, 1, 2$; $x_0 = 1$, $a = 2$, $b = 1$; $L = x_3$.

Planteamiento. por el teorema 5.6, $\lambda_3 = \partial L/\partial x_3 = 1$, $\lambda_t = \lambda_{t+1}\, \partial x_{t+1}/\partial x_t = a\lambda_{t+1}$, y las derivadas respecto a los parámetros suman las contribuciones de cada transición.

Desarrollo.

1. Trayectoria: $x_1 = 3$, $x_2 = 7$, $x_3 = 15$.
2. Multiplicadores: $\lambda_3 = 1$, $\lambda_2 = 2$, $\lambda_1 = 4$.
3. $dL/da = \sum_{t=0}^{2} \lambda_{t+1}\, x_t = \lambda_1 x_0 + \lambda_2 x_1 + \lambda_3 x_2 = 4 + 6 + 7 = 17$.
4. $dL/db = \sum_{t=0}^{2} \lambda_{t+1} \cdot 1 = 4 + 2 + 1 = 7$.
5. Expresión cerrada: $x_3 = a^3 x_0 + b(a^2 + a + 1)$.

Resultado. $x = (1, 3, 7, 15)$**;** $\lambda = (\lambda_1, \lambda_2, \lambda_3) = (4, 2, 1)$**;** $dL/da = 17$ **y** $dL/db = 7$**.**

Comprobación. $dx_3/da = 3a^2 x_0 + b(2a + 1) = 12 + 5 = 17$ y $dx_3/db = a^2 + a + 1 = 7$.

Solución del ejercicio 5.4

Datos. memoria $M(k) = L/k + k$ en unidades de activación; $L = 16$ y $L = 100$.

Planteamiento. minimizar M con derivadas y contar las pasadas como en la proposición 4.10.

Desarrollo.

1. Se guardan L/k puntos de control; al retropropagar un tramo se recalculan y guardan sus k activaciones. La memoria máxima es $M(k) = L/k + k$.
2. $M'(k) = 1 - L/k^2 = 0 \iff k = \sqrt{L}$ (con $k > 0$); $M''(k) = 2L/k^3 > 0$: mínimo, de valor $2\sqrt{L}$.
3. $L = 16$: $k = 4$, $M = 4 + 4 = 8$, frente a 16 sin puntos de control.
4. $L = 100$: $k = 10$, $M = 10 + 10 = 20$, frente a 100.
5. Cómputo: pasada hacia delante ($2N$ FLOPs por token), recálculo ($2N$) y pasada hacia atrás ($4N$): $8N$ por token, un $8/6 - 1 = 33\,\%$ más.

Resultado. $k = \sqrt{L}$ **y memoria** $2\sqrt{L}$**: 8 para** $L = 16$ **y 20 para** $L = 100$**; el paso cuesta** $8N$ **FLOPs por token.**

Comprobación. con $L = 16$, $k = 2$ da $8 + 2 = 10$ y $k = 8$ da $2 + 8 = 10$, ambos mayores que 8.

Solución del ejercicio 5.5

Datos. $N_b = 16$; coste 1 por bloque de activaciones y $c = 1/4$ por punto de control; $M(k) = (N_b/k)c + k$.

Planteamiento. equilibrar la memoria de los puntos de control, decreciente en k, con la del tramo recalculado, creciente, como en el ejercicio 5.4 pero con $c \neq 1$.

Desarrollo.

1. (a) $k = 1$: $16 \cdot 0{,}25 + 1 = 5$. $k = 2$: $8 \cdot 0{,}25 + 2 = 4$. $k = 4$: $4 \cdot 0{,}25 + 4 = 5$. $k = 8$: $2 \cdot 0{,}25 + 8 = 8{,}5$. $k = 16$: $0{,}25 + 16 = 16{,}25$.
2. Óptimo continuo: $M'(k) = -N_b c/k^2 + 1 = 0 \iff k = \sqrt{N_b c} = \sqrt{4} = 2$, con $M(2) = 4$, frente a 16 sin recálculo.
3. (b) Estrategia recursiva: se parte la pila en dos mitades con un punto de control al inicio de cada una; para retropropagar una mitad, se vuelve a partir en dos, y así hasta bloques sueltos. Hay $\log_2 N_b$ niveles.
4. Memoria: en cada nivel se mantiene un número acotado de puntos de control, más las activaciones de un solo bloque: $O(\log N_b)$; con $N_b = 16$, unos $\log_2 16 = 4$ niveles.
5. Cómputo: cada bloque se recalcula una vez por nivel, $O(\log N_b)$ veces, y el total es de $O(N_b \log N_b)$ evaluaciones de bloque, frente a $O(N_b)$ sin recálculo.

Resultado. **(a)** $M = 5$; 4; 5; 8,5; 16,25**, con óptimo** $k = \sqrt{N_b c} = 2$ **y** $M = 4$**. (b) Recálculo recursivo por mitades: memoria** $O(\log N_b)$**, cómputo** $O(N_b \log N_b)$**.**

Comprobación. con $c = 1$ la fórmula se reduce a la del ejercicio 5.4, con óptimo $k = \sqrt{N_b} = 4$.

A.6 Tema 6

Solución del ejercicio 6.1

Datos. bucle de PyTorch sin `opt.zero_grad()`; evaluación con `torch.no_grad()`.

Planteamiento. `backward()` suma en `.grad` (acumulación del tema 3); `no_grad` impide registrar el grafo.

Desarrollo.

1. Sin `zero_grad()`, en el paso t el gradiente guardado es $g_1 + \cdots + g_t$, no g_t.
2. Si los g_i son parecidos, el paso efectivo crece como $t\,\eta\,g$: equivale a multiplicar la tasa de aprendizaje por t, y el entrenamiento se vuelve errático o diverge.
3. `torch.no_grad()` no crea nodos ni guarda tensores. Al evaluar no hace falta derivar, así que ahorra memoria y tiempo, y la evaluación no deja rastro en `.grad` ni en el grafo del paso siguiente.

Resultado. **sin `zero_grad()` los gradientes se acumulan entre pasos; `no_grad()` evalúa sin registrar el grafo.**

Comprobación. en el ejercicio 6.2, dos llamadas a `backward()` sobre $y = x^2$ en $x = 3$ dejan `x.grad` $= 6 + 6 = 12$.

Solución del ejercicio 6.2

Datos. $x = 3$, $y = x \cdot x$, dos llamadas a `backward()`.

Planteamiento. PyTorch libera los tensores guardados tras la pasada hacia atrás, salvo con `retain_graph=True`; `.grad` se acumula.

Desarrollo.

1. (a) Primera llamada: `x.grad` $= 0 + 2 \cdot 3 = 6$, y se liberan los valores guardados del nodo producto.
2. Segunda llamada: el VJP del producto necesita esos valores, que ya no existen, y PyTorch lanza un `RuntimeError` que indica que se intenta retropropagar por el grafo una segunda vez.
3. (b) Con `retain_graph=True` el grafo se conserva: tras la primera llamada `x.grad` $= 6$ y tras la segunda `x.grad` $= 6 + 6 = 12$.

Resultado. **(a) error en la segunda llamada; (b)** `x.grad` $= 12$, el doble del gradiente, por la acumulación.

Comprobación. es el mismo efecto que olvidar `zero_grad()` (ejercicio 6.1). PyTorch 2.14 reproduce ambos comportamientos.

Solución del ejercicio 6.3

Datos. operaciones $a - b$, $a \cdot b$, a/b, $\ln a$, $\exp a$, AB; $f = \ln(a/b) - \exp(a \cdot b)$ en $(2, 1)$.

Planteamiento. cada función hacia atrás aplica la regla local (VJP) y devuelve un adjunto por entrada; la cinta se recorre al revés (observación 3.10).

Desarrollo.

1. Reglas: $a - b$: $(g, -g)$. $a \cdot b$: $(g\,b,\ g\,a)$. a/b: $(g/b,\ -g\,a/b^2)$. $\ln a$: g/a. $\exp a$: $g\,e^a$. AB: $(G\,B^\top,\ A^\top G)$ (proposición 4.6).
2. Cinta hacia delante: $v_1 = a/b = 2$; $v_2 = \ln v_1 = 0{,}6931$; $v_3 = a \cdot b = 2$; $v_4 = \exp v_3 = 7{,}389$; $f = v_2 - v_4 = -6{,}696$.
3. Hacia atrás: resta, $\bar{v}_2 = 1$, $\bar{v}_4 = -1$; exponencial, $\bar{v}_3 = -1 \cdot e^2 = -7{,}389$; logaritmo, $\bar{v}_1 = 1/2 = 0{,}5$.
4. Producto $v_3 = a \cdot b$: $\bar{a}$ += $-7{,}389 \cdot 1$ y $\bar{b}$ += $-7{,}389 \cdot 2 = -14{,}78$.
5. Cociente $v_1 = a/b$: $\bar{a}$ += $0{,}5/1$, luego $\bar{a} = -6{,}889$; $\bar{b}$ += $-0{,}5 \cdot 2/1^2 = -1$, luego $\bar{b} = -15{,}78$.

Resultado. $f(2, 1) \approx -6{,}696$**,** $\partial f/\partial a \approx -6{,}889$ **y** $\partial f/\partial b \approx -15{,}78$**.**

Comprobación. $f = \ln a - \ln b - e^{ab}$: $\partial_a f = 1/a - b\,e^{ab} = 0{,}5 - 7{,}389 = -6{,}889$ y $\partial_b f = -1/b - a\,e^{ab} = -1 - 14{,}78 = -15{,}78$.

Solución del ejercicio 6.4

Datos. $V = 28$, $d = 8$, $B = 3$, $H = 64$.

Planteamiento. embeddings: $V \cdot d$; capa lineal de n entradas y k salidas: nk pesos y k sesgos.

Desarrollo.

1. Embeddings: $28 \cdot 8 = 224$.

2. Capa oculta: entrada $B \cdot d = 24$; $24 \cdot 64 + 64 = 1536 + 64 = 1600$.
3. Capa de salida: $64 \cdot 28 + 28 = 1792 + 28 = 1820$.
4. Total: $224 + 1600 + 1820 = 3644$.

Resultado. $224 + 1600 + 1820 = 3644$ **parámetros.**

Comprobación. con $d = 10$ se obtiene $280 + 1984 + 1820 = 4084$, la cifra del ejemplo 6.5.

Solución del ejercicio 6.5

Datos. $o = \tanh(wx + b)$, $x = 1$, $w = 0{,}5$, $b = 0$, $y = 1$, $L = \frac{1}{2}(o - y)^2$, $\eta = 0{,}5$.

Planteamiento. modo inverso por los nodos $n = wx + b$, $o = \tanh n$ y L; después, $w \leftarrow w - \eta\,\bar{w}$ y $b \leftarrow b - \eta\,\bar{b}$.

Desarrollo.

1. $n = 0{,}5$; $o = 0{,}46212$; $L = \frac{1}{2}(0{,}46212 - 1)^2 = 0{,}14466$.
2. $\bar{o} = o - y = -0{,}53788$; $\bar{n} = \bar{o}\,(1 - o^2) = -0{,}53788 \cdot 0{,}78645 = -0{,}42302$.
3. $\bar{w} = \bar{n}\,x = -0{,}4230$; $\bar{b} = \bar{n} = -0{,}4230$.
4. Paso: $w = 0{,}5 + 0{,}5 \cdot 0{,}42302 = 0{,}71151$; $b = 0 + 0{,}21151 = 0{,}21151$.
5. Nueva pérdida: $n = 0{,}92302$, $o = \tanh 0{,}92302 = 0{,}72732$ y $L = \frac{1}{2}(0{,}72732 - 1)^2 = 0{,}03718$.

Resultado. $L = 0{,}1447$**,** $\partial L/\partial w = \partial L/\partial b \approx -0{,}4230$**; tras el paso,** $w \approx 0{,}7115$**,** $b \approx 0{,}2115$ **y** $L \approx 0{,}0372$**.**

Comprobación. la pérdida disminuye de 0,1447 a 0,0372. El mismo paso en PyTorch, con `torch.no_grad()` en la actualización, da los mismos números.

Solución del ejercicio 6.6

Datos. $\ell_s(w) = \frac{1}{2}(wx_s - y_s)^2$; $x = (1, 2, 3, 4)$, $y = (2, 3, 7, 8)$, $w = 1$; $k = 2$ microlotes, $S_1 = \{1, 2\}$ y $S_2 = \{3, 4\}$.

Planteamiento. linealidad de la derivada (proposición 6.7); $\ell_s'(w) = (wx_s - y_s)x_s$.

Desarrollo.

1. (a) Tras las k llamadas, el acumulador vale $\sum_r \nabla(\mathcal{L}_r/k) = \nabla\big(\frac{1}{k}\sum_r \mathcal{L}_r\big)$, y con $|S_r| = |S|/k$ se tiene $\frac{1}{k}\sum_r \mathcal{L}_r = \mathcal{L}$.
2. Sin dividir por k, el acumulador vale $\sum_r \nabla\mathcal{L}_r = k\,\nabla\mathcal{L}$: el gradiente queda multiplicado por k, como si la tasa de aprendizaje fuera k veces mayor.
3. (b) Residuos $wx_s - y_s = (-1, -1, -4, -4)$; derivadas $\ell_s' = (-1, -2, -12, -16)$.
4. Lote completo: $\nabla\mathcal{L} = (-1 - 2 - 12 - 16)/4 = -31/4 = -7{,}75$.
5. Microlote 1: $\nabla\mathcal{L}_1 = (-1 - 2)/2 = -1{,}5$; acumulado tras dividir por 2: $-0{,}75$.
6. Microlote 2: $\nabla\mathcal{L}_2 = (-12 - 16)/2 = -14$; se suma -7: acumulado $-7{,}75$.

Resultado. **el gradiente acumulado es** $-7{,}75$**, igual al del lote completo**; sin dividir por k sería $-15{,}5$.

Comprobación. PyTorch, con dos llamadas a `backward()` sin `zero_grad()` intermedio, deja `w.grad` $= -0{,}75$ y después $-7{,}75$.

Bibliografía

[1] W. Baur y V. Strassen. The complexity of partial derivatives. *Theoretical Computer Science*, 22(3):317–330, 1983.

[2] A. G. Baydin, B. A. Pearlmutter, A. A. Radul y J. M. Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018. https://jmlr.org/papers/v18/17-468.html

[3] T. Chen, B. Xu, C. Zhang y C. Guestrin. Training deep nets with sublinear memory cost. arXiv:1604.06174, 2016. https://arxiv.org/abs/1604.06174

[4] B. Raj y equipo docente de 11-785. 11-785: Introduction to Deep Learning. Tarea 1, parte complementaria, «An Introduction to Autograd». Carnegie Mellon University, otoño de 2021. https://github.com/CMU-IDeeL/new_grad

[5] A. Karpathy y equipo docente de CS231n. CS231n: Deep Learning for Computer Vision. Notas del curso, «Backpropagation, Intuitions». Stanford University. https://cs231n.github.io/optimization-2/

[6] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 1, «Basics», versión 26.0.3. Stanford University, primavera de 2026. https://github.com/stanford-cs336/assignment1-basics

[7] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 2, «Systems», versión 26.1.3. Stanford University, primavera de 2026. https://github.com/stanford-cs336/assignment2-systems

[8] Gemini Team, Google. Gemini: a family of highly capable multimodal models. arXiv:2312.11805, 2023. https://arxiv.org/abs/2312.11805

[9] I. Goodfellow, Y. Bengio y A. Courville. *Deep Learning*. MIT Press, 2016. §6.5, «Back-Propagation and Other Differentiation Algorithms». https://www.deeplearningbook.org/

[10] A. Griewank y A. Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2.ª ed. SIAM, 2008.

[11] xAI. Grok-1: código de ejemplo en JAX para cargar y ejecutar el modelo de pesos abiertos. 2024. https://github.com/xai-org/grok-1

[12] R. Grosse. Lecture 6: Automatic Differentiation. Notas del curso CSC421/2516, «Neural Networks and Deep Learning». University of Toronto, 2019. https://www.cs.toronto.edu/~rgrosse/courses/csc421_2019/

[13] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu y D. Amodei. Scaling laws for neural language models. arXiv:2001.08361, 2020. https://arxiv.org/abs/2001.08361

[14] A. Karpathy. micrograd: un pequeño motor de diferenciación automática en modo inverso sobre un grafo construido dinámicamente. 2020. https://github.com/karpathy/micrograd

[15] S. Linnainmaa. The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors. Tesis de maestría, Universidad de Helsinki, 1970.

[16] Meta. Llama 2: código de inferencia de referencia (PyTorch). 2023. https://github.com/meta-llama/llama

[17] C. Rackauckas. Parallel Computing and Scientific Machine Learning (18.337). Notas de la clase 8, «Forward-Mode Automatic Differentiation (AD) via High Dimensional Algebras». Massachusetts Institute of Technology, 2020. https://book.sciml.ai/

[18] A. Edelman y S. G. Johnson. 18.S096: Matrix Calculus for Machine Learning and Beyond. Clase 5, partes 2 y 3. Massachusetts Institute of Technology, MIT OpenCourseWare, IAP de enero de 2023. https://ocw.mit.edu/courses/18-s096-matrix-calculus-for-machine-learning-and-beyond-january-iap-2023/

[19] A. Paszke *et al.* PyTorch: an imperative style, high-performance deep learning library. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. https://arxiv.org/abs/1912.01703

[20] B. A. Pearlmutter. Fast exact multiplication by the Hessian. *Neural Computation*, 6(1):147–160, 1994.

[21] PyTorch. Autograd mechanics. Documentación de PyTorch, versión 2.14. https://docs.pytorch.org/docs/stable/notes/autograd.html

[22] D. E. Rumelhart, G. E. Hinton y R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[23] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. §2.5, «Automatic Differentiation», y §5.3, «Forward Propagation, Backward Propagation, and Computational Graphs». https://d2l.ai/